MEKDELA AMBA UNIVERSITY



College of computing and informatics

Department of Software Engineering

Course name: Software Architecture&Design

Project title: Smart Hospital Management System(SHMS)

Prepared by group five(5)

Name of Students: ID

1) Pel Joang Kim ……..……1601018
2) Demozu Wedaj ……..1600409
3) Baye Mulu …….1600225

Submitted to : Ms Destaye A

Submission date: **April 02, 2018G.C**

CHAPTER 1. INTRODUCTION....4
1.1 Purpose.... 4
1.2 Scope....4
1.3 Overview....5
1.4 Intended Users.... 6
1.5 Reference Material.... 6
1.6 Definitions, Abbreviations and Acronyms.... 7
CHAPTER 2. SYSTEM OVERVIEW AND CONSTRAINTS.... 7
2.1 System Overview....8
2.2 Constraints....13
CHAPTER 3. QUALITY ATTRIBUTE SCENARIOS AND TACTIC SELECTION....16
3.1 Quality Attribute Scenarios.... 16
1: PERFORMANCE....16
QA-2: AVAILABILITY.... 17
QA-3: SECURITY.... 18
QA-4: SCALABILITY....19
QA-5: MAINTAINABILITY.... 19
3.2 Tactic Selection....21
CHAPTER 4. SYSTEM ARCHITECTURE.... 22
4.1 Architectural Design....22
Architecture Pattern: Layered (N-Tier) Architecture....22
4.1 Architectural Design....23
Subsystem Decomposition.... 23
4.2 Module View.... 26
Class Diagram (Key Entities)....26
4.3 Component and Connector View....29
Sequence Diagram: Patient Registration....29
4.4 Allocation View.... 31
Deployment Diagram.... 34
4.5 Design Rationale.... 38
4.6 Design Rationale.... 43
CHAPTER 5. DATA DESIGN.... 45
5.1 Data Description.... 45
5.2 Data Dictionary.... 47
CHAPTER 6. COMPONENT DESIGN.... 51
6.1 Use Case Diagram.... 51
6.2 Component Algorithms (PDL/Pseudocode)....60
This subsection provides the procedural logic for key service methods using Procedural Description Language (PDL). This ensures developers understand the exact flow, error handling, and data interactions....60

Component: AppointmentService..... 60
Method: bookAppointment(AppointmentRequest req).....60
Component: BillingService.....62
Method: generateInvoice(InvoiceRequest req).....62
Component: MedicalRecordService..... 63
6.3 Deployment Diagram.....64
A. Deployment Diagram Description.....64
6.4 Environment Description.....67
B. Software Requirements.....68
C. Network Configuration.....69
Security Groups / Firewall Rules:..... 69
1. DB Security Group: Allow Port 5432 ONLY from 10.0.2.0/12 (App Network)..... 69
2. App Security Group: Allow Port 8080 ONLY from 10.0.1.0/12 (DMZ).....69
3. Redis Security Group: Allow Port 6379 ONLY from 10.0.2.0/12..... 69
4. SSH Access: Restricted to Bastion Host in Management Network.....69
CHAPTER 7. REQUIREMENTS MATRIX.....69
7.1 Traceability Matrix (SRS → Architecture)..... 69
7.2 Design Decisions & Trade-offs..... 71
Decision 1: Architectural Style..... 71
Decision 2: Database Technology.....72
Decision 3: Authentication Mechanism.....72
Decision 4: Communication Pattern.....73
Decision 5: Caching Strategy.....73
Decision 6: Deployment Environment.....74

❖ List of tables

Audience Role & Usage ..... ……4
Goals & Benefits ..... ……..5
Intended Users 6
Reference Material 6
Definitions & Acronyms..... 7
Functional Modules.....11
Technical Constraints 13
Development Tools Constraints ..... ……13
Legacy System Constraints ..... ……15
Regulatory & Compliance Constraints 15
Performance Constraints 15
Quality Attribute Scenarios 16–20
Tactic Selection 21
Subsystem Decomposition ..... ……..23
Why Layered Architecture (Advantages) 25

Hardware Requirements ..... ……40
Environment Description ..... ……40
Software Requirements ..... ……41
Network Segments ..... ……42
Design Rationale (Architecture Pattern Comparison) ..... ……43
Backend Framework Decision (Spring Boot vs Others) ..... ……44
Data Types & Storage ..... ……45
Data Dictionary – User Table ..... ……49
Data Dictionary – Patients Table ..... ……..50
Data Dictionary – Doctors Table ..... ……50
Data Dictionary – Appointments Table ..... ……50–51
Use Case 1 – Register Patient 53
Use Case 2 – Book Appointment ..... ……54
Use Case 3 – Create Medical Record ..... ……56
Use Case 4 – Process Payment ..... ……..57
Use Case 5 – Order Laboratory Test 59
Physical Node Mapping 67
Network Configuration 69
Traceability Matrix ..... ……70
Design Decisions Summary ..... ……77

List of Figures

System Context Diagram 9

System Architecture Overview 10

Project Team Structure 14

Layered Architecture Design 23

Subsystem Interaction Diagram 24

N-Tier Architecture Detail 25

Module Decomposition 26

Class Diagram (Key Entities) 27

Package Diagram 28

Sequence Diagram - Patient Registration 29

Sequence Diagram - Appointment Booking 30
Component & Connector View 31
High-Level Architecture 32
UML Component Diagram 33
Subsystem Interaction Diagram 34
Component Diagram 35
Subsystem Interaction (Data Flow) 36
Deployment Diagram 37
Physical Deployment View 38
Network Configuration Diagram 42
Data Organization 46
Data Dictionary Diagram 47
Data Dictionary Header 48
Entity Relationship Diagram (ERD) 49
Use Case Diagram 52
PDL - AppointmentService (Page 1) 61
PDL - AppointmentService (Page 2) 62
PDL - BillingService 63
PDL - MedicalRecordService 64
Deployment Diagram Description 65
Kubernetes Deployment Architecture 66

CHAPTER 1. INTRODUCTION

1.1 Purpose

The purpose of this Software Design Document (SDD) is to describe the complete architecture, system design, and implementation strategy for the Hospital Management System (HMS). This document serves as the primary reference for:

- Software Architects: To validate architectural decisions and ensure alignment with quality attributes.
- Developers: To provide coding guidelines, component interfaces, and data structures.
- QA Team: To derive test cases based on functional flows and non-functional requirements.
- Stakeholders: To understand the system's capabilities, security measures, and scalability potential.

Audience	Role	Usage
Software Architects	System Design	Architecture validation and design decisions
Developers	Implementation	Coding guidelines and component development
Project Managers	Planning	Timeline and resource allocation
QA Team	Testing	Test case development and validation
Database Administrators	Data Management	Database design and optimization
Stakeholders	Review	System understanding and approval

1.2 Scope

System Description:
The Hospital Management System is a comprehensive, web-based application designed to automate and streamline hospital operations including patient

management, appointment scheduling, medical records, billing, inventory management, and staff administration.

Goal	Objective	Benefit
Automation	Digitize manual processes	80% reduction in paperwork
Efficiency	Streamline workflows	50% faster patient processing
Data Management	Centralized patient records	Instant access to medical history
Integration	Connect all departments	Seamless inter-department communication
Security	Protect sensitive data	HIPAA compliance
Scalability	Support growth	Handle 10,000+ patients

- Scope Boundaries:

➢ In Scope:

- Patient registration and management
- Appointment scheduling
- Electronic Medical Records (EMR)
- Billing and invoicing
- Pharmacy and inventory management
- Staff management
- Laboratory management
- Reporting and analytics

➢ Out of Scope:

- Telemedicine features (Phase 2)
- Mobile application (Phase 2)
- AI-based diagnosis (Future enhancement)

1.3 Overview

This document is organized into seven major sections:

1. Introduction: Purpose, scope, and definitions.
2. System Overview: Functional context and technical/regulatory constraints.
3. Quality Attributes: Scenarios and architectural tactics for Performance, Security, Availability, etc.
4. System Architecture: High-level design, module decomposition, and component interactions.

5. Data Design: Database schema, ERD, and data dictionary.
6. Component Design: Detailed use cases, deployment topology, and algorithmic logic.
7. Requirements Matrix: Traceability from SRS to architectural components.

1.4 Intended Users

User Category	Role Description	System Access Level
Administrators	Hospital management	Full system configuration, user management, analytics
Doctors	Medical practitioners	View/Edit EMR, Prescribe meds, Order labs
Nurses	Nursing staff	Update vitals, Manage patient care plans
Receptionists	Front desk staff	Patient registration, Appointment scheduling
Pharmacists	Pharmacy staff	Dispense meds, Manage inventory
Lab Technicians	Laboratory staff	Enter test results, Manage lab workflows
Patients	End-users (Portal)	View own records, Book appointments, Pay bills
Accountants	Billing staff	Generate invoices, Process insurance claims

1.5 Reference Material

Document	Version	Purpose
Software Requirements Specification (SRS)	1	Functional requirements
HIPAA Compliance Guidelines	2024	Security standards
HL7 FHIR Standards	R4	Health data interchange
IEEE 830-1998	-	SDD Standard
Hospital Workflow Analysis Report	1	Business process documentation

1.6 Definitions, Abbreviations and Acronyms

Term/Acronym	Definition
HMS	Hospital Management System
EMR	Electronic Medical Records
EHR	Electronic Health Records
HIPAA	Health Insurance Portability and Accountability Act
HL7	Health Level Seven International
FHIR	Fast Healthcare Interoperability Resources
API	Application Programming Interface
UI/UX	User Interface/User Experience
DBMS	Database Management System
RBAC	Role-Based Access Control
SSL/TLS	Secure Sockets Layer/Transport Layer Security
OTP	One-Time Password
PDF	Portable Document Format
SMS	Short Message Service
UI	User Interface

CHAPTER 2. SYSTEM OVERVIEW AND CONSTRAINTS

2.1 System Overview

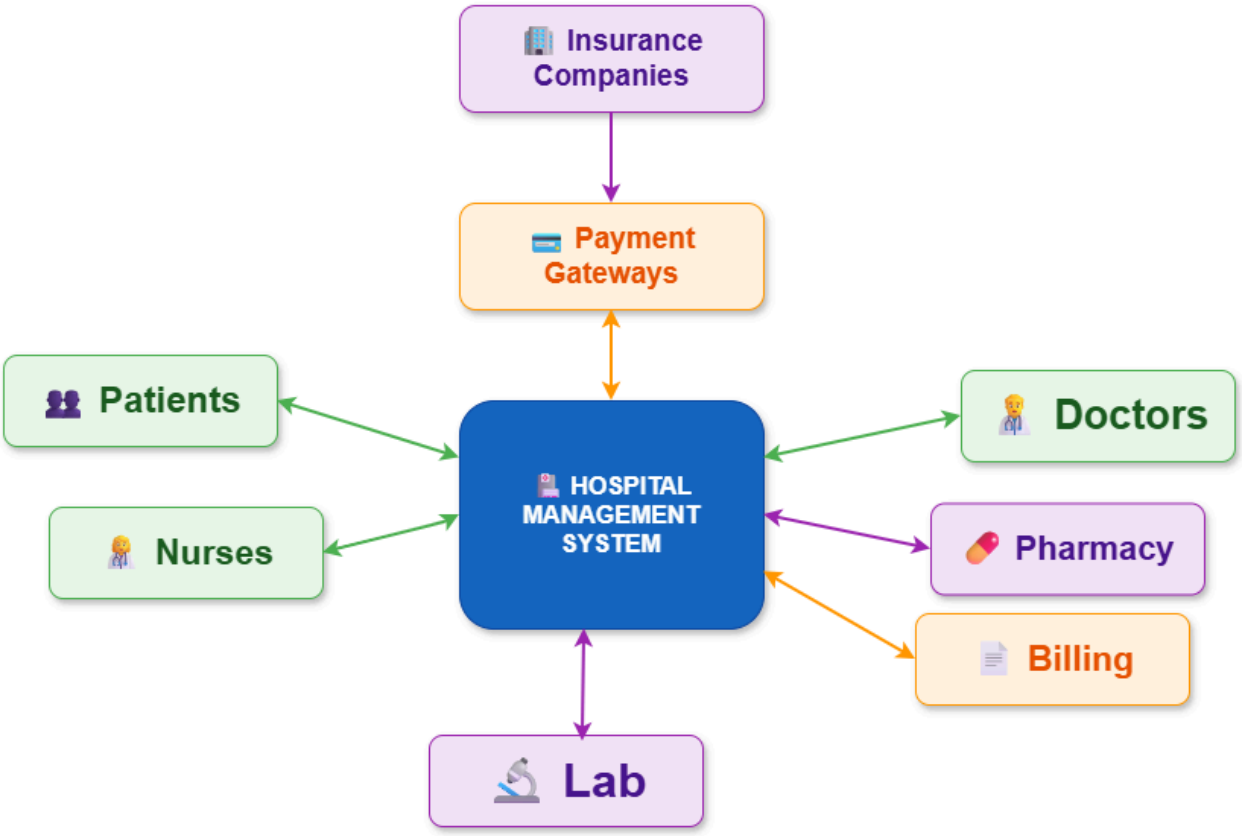
The HMS operates as a centralized platform connecting various hospital departments. The system context involves internal users (staff) accessing via a secure web portal and external users (patients) accessing via a public-facing portal.
High-Level Functional Modules:

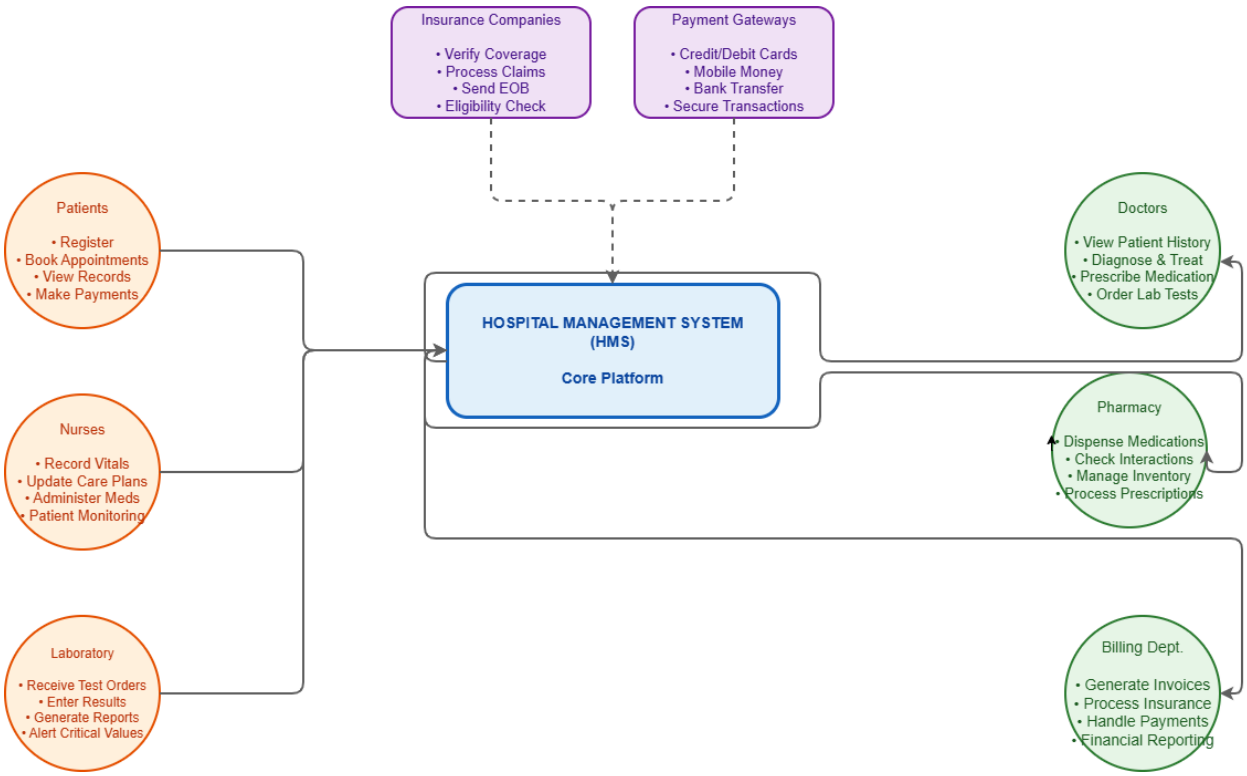
1. Patient Management: Registration, demographic updates, insurance verification.
2. Appointment Scheduling: Real-time slot booking, doctor availability management.
3. Clinical Services (EMR): Diagnosis, prescriptions, vital signs, medical history.
4. Laboratory Management: Test ordering, result entry, report generation.
5. Pharmacy: Drug inventory, prescription fulfillment, stock alerts.
6. Billing & Finance: Invoice generation, payment processing, insurance claim submission.
7. Staff Management: HR records, shift scheduling, payroll integration.
8. Reporting: Analytics dashboards for hospital administratio

System Context Diagram:

System Context Diagram

2.1 System Overview

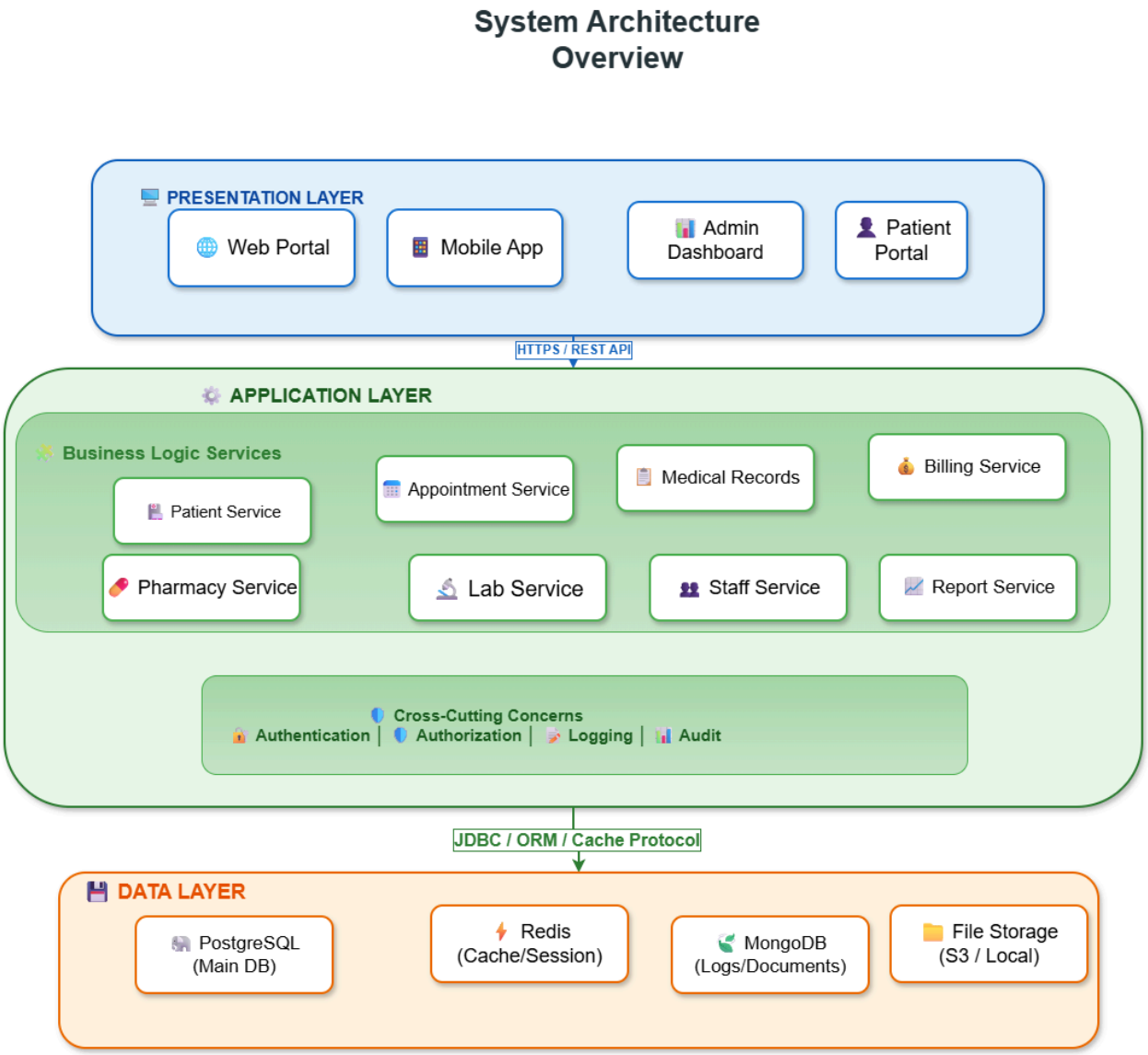



Insurance Companies
• Verify Coverage
• Process Claims
• Send EOB
• Eligibility Check
Payment Gateways
• Credit/Debit Cards
• Mobile Money
• Bank Transfer
• Secure Transactions
Patients
• Register
• Book Appointments
• View Records
• Make Payments
HOSPITAL MANAGEMENT SYSTEM (HMS)
Core Platform
Doctors
• View Patient History
• Diagnose & Treat
• Prescribe Medication
• Order Lab Tests
Nurses
• Record Vitals
• Update Care Plans
• Administer Meds
• Patient Monitoring
Pharmacy
• Dispense Medications
• Check Interactions
• Manage Inventory
• Process Prescriptions
Laboratory
• Receive Test Orders
• Enter Results
• Generate Reports
• Alert Critical Values
Billing Dept.
• Generate Invoices
• Process Insurance
• Handle Payments
• Financial Reporting

Functional Description:
The Hospital Management System provides comprehensive functionality across multiple modules:

Module	Primary Functions	Users
Patient Management	Registration, Profile management, Medical history	Receptionists, Patients
Appointment Scheduling	Book, reschedule, cancel appointments	Receptionists, Patients, Doctors
Electronic Medical Records	Create, update, view patient records	Doctors, Nurses
Prescription Management	Generate prescriptions, medication history	Doctors, Pharmacists
Laboratory Management	Test ordering, result entry, reports	Lab Technicians, Doctors
Pharmacy Management	Inventory, dispensing, stock alerts	Pharmacists
Billing & Invoicing	Generate bills, process payments, insurance claims	Accountants, Receptionists
Staff Management	Employee records, scheduling, payroll	Administrators
Inventory Management	Medical supplies tracking, reorder alerts	Administrators, Pharmacists
Reporting & Analytics	Generate reports, dashboards, statistics	Administrators, Management

System Architecture Overview:



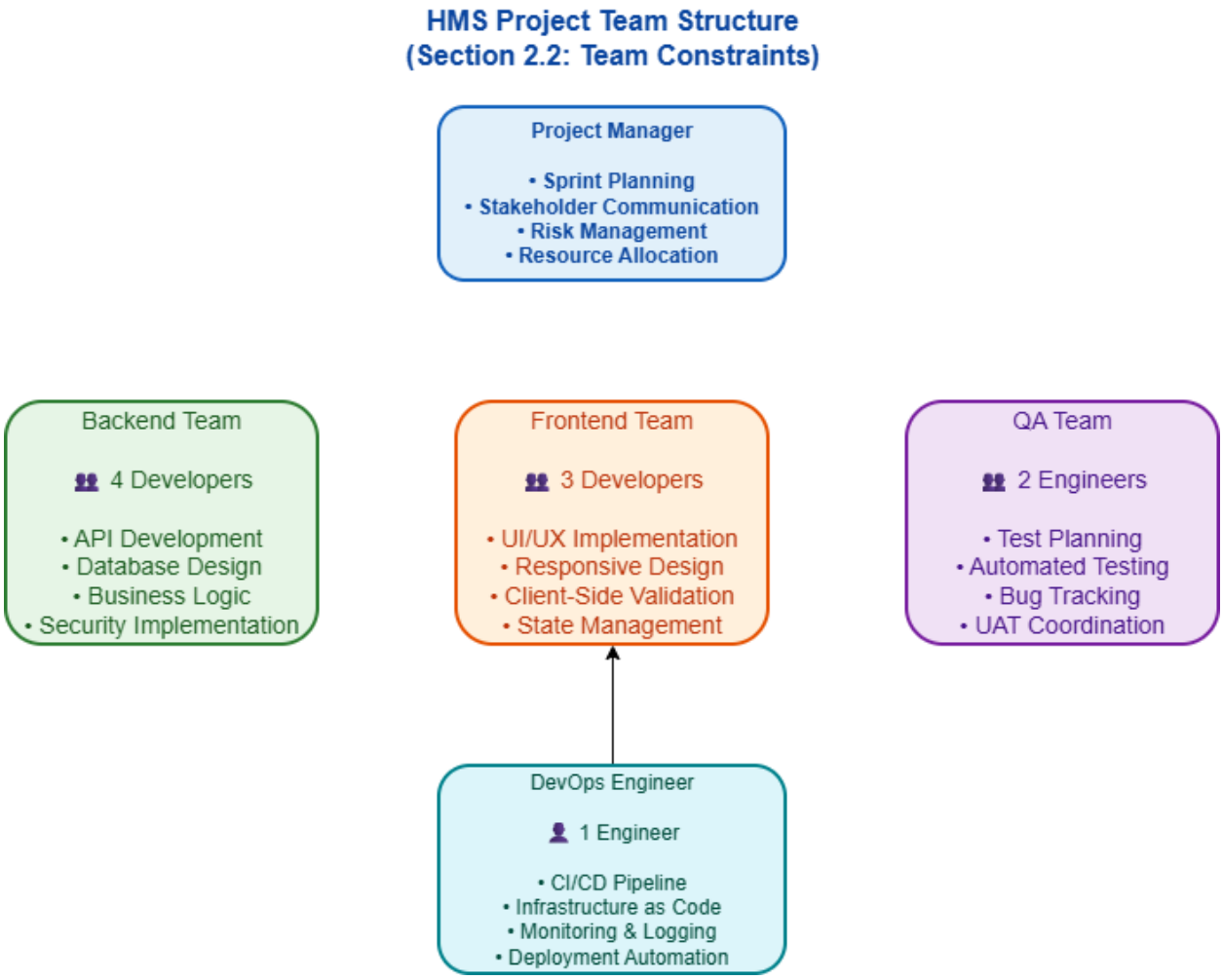
2.2 Constraints

Technical Constraints:

Constraint	Description	Rationale
Architecture Pattern	Layered Architecture with Microservices readiness	Scalability and maintainability
API Standard	RESTful API with JSON	Industry standard, interoperability
Authentication	JWT (JSON Web Tokens) + OAuth 2.0	Stateless, secure authentication
Data Validation	Server-side + Client-side validation	Data integrity and security
Error Handling	Global exception handling with logging	Consistent error responses
Code Quality	SonarQube analysis, 80% code coverage	Maintainable, bug-free code

Development Tools Constraints:

Tool Category	Tool Name	Version	Purpose
IDE	IntelliJ IDEA / VS Code	Latest	Development environment
Version Control	Git	2.35+	Source code management
Build Tool	Maven / Gradle	Latest	Dependency management, build
CI/CD	Jenkins / GitHub Actions	Latest	Automated builds and deployment
Containerization	Docker	20.10+	Application containerization
Orchestration	Kubernetes	1.24+	Container orchestration
Testing	JUnit 5, Selenium	Latest	Unit and integration testing
API Testing	Postman / Insomnia	Latest	API development and testing


HMS Project Team Structure
(Section 2.2: Team Constraints)
Project Manager
• Sprint Planning
• Stakeholder Communication
• Risk Management
• Resource Allocation
Backend Team
4 Developers
• API Development
• Database Design
• Business Logic
• Security Implementation
Frontend Team
3 Developers
• UI/UX Implementation
• Responsive Design
• Client-Side Validation
• State Management
QA Team
2 Engineers
• Test Planning
• Automated Testing
• Bug Tracking
• UAT Coordination
DevOps Engineer
1 Engineer
• CI/CD Pipeline
• Infrastructure as Code
• Monitoring & Logging
• Deployment Automation

❖ Legacy System Constraints

Legacy System	Integration Requirement	Challenge	Solution
Existing Patient Database	Data migration	Different schema	ETL process with mapping
Old Billing System	Parallel run for 3 months	Data sync	API-based integration
Laboratory Equipment	HL7 message integration	Proprietary protocols	HL7 FHIR adapter

❖ Regulatory & Compliance Constraints:

Regulation	Requirement	Implementation
HIPAA	Patient data privacy	Encryption, access controls, audit logs
GDPR	Data protection (if EU patients)	Right to erasure, consent management
HL7 FHIR	Health data interoperability	FHIR-compliant APIs
SOC 2	Security controls	Regular audits, monitoring

❖ Performance Constraints:

Metric	Target	Measurement
Response Time	< 2 seconds for 95% requests	Application monitoring
Concurrent Users	Support 500+ simultaneous users	Load testing
Database Queries	< 100ms for standard queries	Query optimization
API Availability	99.9% uptime	SLA monitoring
Data Backup	Daily backups, RPO < 24hrs	Backup verification

CHAPTER 3. QUALITY ATTRIBUTE SCENARIOS AND TACTIC SELECTION

3.1 Quality Attribute Scenarios

Scenario Template:

- Source: Stimulus origin
- Stimulus: Event that arrives
- Environment: Conditions when stimulus occurs
- Artifact: System component affected
- Response: Action taken by system
- Response Measure: Quantifiable metric

1: PERFORMANCE

Scenario 1.1: Patient Record Retrieval

Attribute	Description
Quality Attribute	Performance
Source	Doctor/Nurse accessing patient records
Stimulus	Request to retrieve complete patient medical history
Environment	Normal operation with 200 concurrent users
Artifact	Medical Records Service, Database
Response	Fetch and display patient records with all historical data
Response Measure	95% of requests complete within 2 seconds

Scenario 1.2: Appointment Booking Peak Load

Attribute	Description
Quality Attribute	Performance
Source	Multiple patients booking appointments simultaneously
Stimulus	100 appointment booking requests within 1 minute
Environment	Monday morning peak hours
Artifact	Appointment Service, Booking Engine
Response	Process all booking requests without system degradation
Response Measure	System maintains < 3 second response time, zero failed transactions

QA-2: AVAILABILITY

Scenario 2.1: Database Server Failure

Attribute	Description
Quality Attribute	Availability
Source	Database server hardware failure
Stimulus	Primary database becomes unresponsive
Environment	System handling 300 active users
Artifact	Database cluster, Application servers
Response	Automatic failover to standby database, minimal service disruption
Response Measure	System recovers within 30 seconds, data loss < 5 seconds of transactions

Scenario 2.2: Scheduled Maintenance

Attribute	Description
Quality Attribute	Availability
Source	System administrators
Stimulus	Scheduled maintenance window notification
Environment	Low-traffic period (2 AM - 4 AM)
Artifact	All system components
Response	Graceful shutdown, maintenance execution, system restart
Response Measure	Maximum downtime 15 minutes, 99.9% monthly uptime

QA-3: SECURITY

Scenario ID	QA-3.1: Unauthorized Access Attemp
Source	Malicious actor (external hacker) or compromised internal account.
Stimulus	Attempt to access patient records without proper authorization (e.g., Doctor A trying to view Patient B without referral).
Environment	Normal system operation; Valid JWT token present but insufficient privileges.
Artifact	SecurityService, Authorization Module, Audit

	Logger.
Response	Access denied (HTTP 403); Incident logged with IP/User ID; Security team alerted if pattern detected.
Response Measure	Access blocked immediately. Alert generated within 1 minute. Account locked after 5 failed attempts

QA-4: SCALABILITY

Scenario ID	QA-4.1: User Base Growth (Horizontal Scaling)
Source	Hospital expansion (merger with another clinic) or seasonal flu outbreak.
Stimulus	Concurrent user base grows from 1,000 to 10,000 over 6 months.
Environment	Production environment; Cloud infrastructure (AWS/Azure).
Artifact	Kubernetes Cluster, Application Pods, Load Balancer.
Response	Auto-scaling group triggers; New application instances spun up; Load balanced across new nodes.
Response Measure	System scales linearly. Response time remains < 3 seconds despite 10x load increase.

QA-5: MAINTAINABILITY

Scenario ID	QA-5.1: Critical Bug Fix Deploymen
Source	QA Team / Production Monitoring Alert.
Stimulus	Critical bug identified in Billing Module (e.g., incorrect tax calculation).
Environment	Live production system; High transaction volume.
Artifact	Billing Service, CI/CD Pipeline, Kubernetes Deployment.
Response	Fix developed; Automated tests passed; Blue-Green deployment executed; Traffic switched to new version.
Response Measure	Zero downtime during deployment. Fix live within 24 hours of detection.

Scenario 3.1: Unauthorized Access Attempt

Attribute	Description
Quality Attribute	Security
Source	External attacker or unauthorized user
Stimulus	Multiple failed login attempts (5+ within 2 minutes)
Environment	Any time, from suspicious IP address
Artifact	Authentication Service, User Account

Response	Account temporary lockout, IP blocking, security alert to admin

Scenario ID	QA-6.1: Power Failure During Transaction
Source	Infrastructure Failure (Power Outage / Server Crash).
Stimulus	Server loses power while processing a patient admission payment.
Environment	Active database transaction; Uncommitted data in memory.
Artifact	Database Transaction Manager, UPS (Uninterruptible Power Supply).
Response	Transaction rolled back automatically upon recovery; Data remains consistent; User prompted to retry.
Response Measure	Zero data loss (ACID compliance). No orphaned records.

3.2 Tactic Selection

Quality Attribute	Design Concern	Tactic Selected	Justification	Implementation Approach
Quality Attribute	**Design Concern**	**Tactic Selected**	**Justification**	**Implementation Approach**
PERFORMANCE	Response Time	Caching	Reduces DB load	Redis cache for patient profiles & appointment slots.

PERFORMANCE	Throughput	Connection Pooling	Reuses DB connections	HikariCP (Max 50 connections).
AVAILABILITY	Fault Recovery	Redundancy & Failover	Backup components ready	DB Replication (Primary-Standby) , K8s Pod Auto-restart.
AVAILABILITY	Graceful Degradation	Circuit Breaker	Prevent cascade failures	Resilience4j pattern for external service calls.
SECURITY	Authentication	MFA & JWT	Strong identity verification	JWT + OTP via SMS/Email.
SECURITY	Data Confidentiality	Encryption	Protect sensitive data	AES-256 at rest, TLS 1.3 in transit.
SCALABILITY	Horizontal Scaling	Containerization	Independent service scaling	Docker containers orchestrated by Kubernetes.
MAINTAINABILITY	Modifiability	Modular Design	Easy to modify components	Clean Architecture, Separation of Concerns.
RELIABILITY	Data Integrity	ACID Transactions	Ensure consistency	PostgreSQL Transaction Management.
USABILITY	Learnability	Responsive UI	Intuitive interface	React with Material-UI, WCAG 2.1

CHAPTER 4. SYSTEM ARCHITECTURE

4.1 Architectural Design

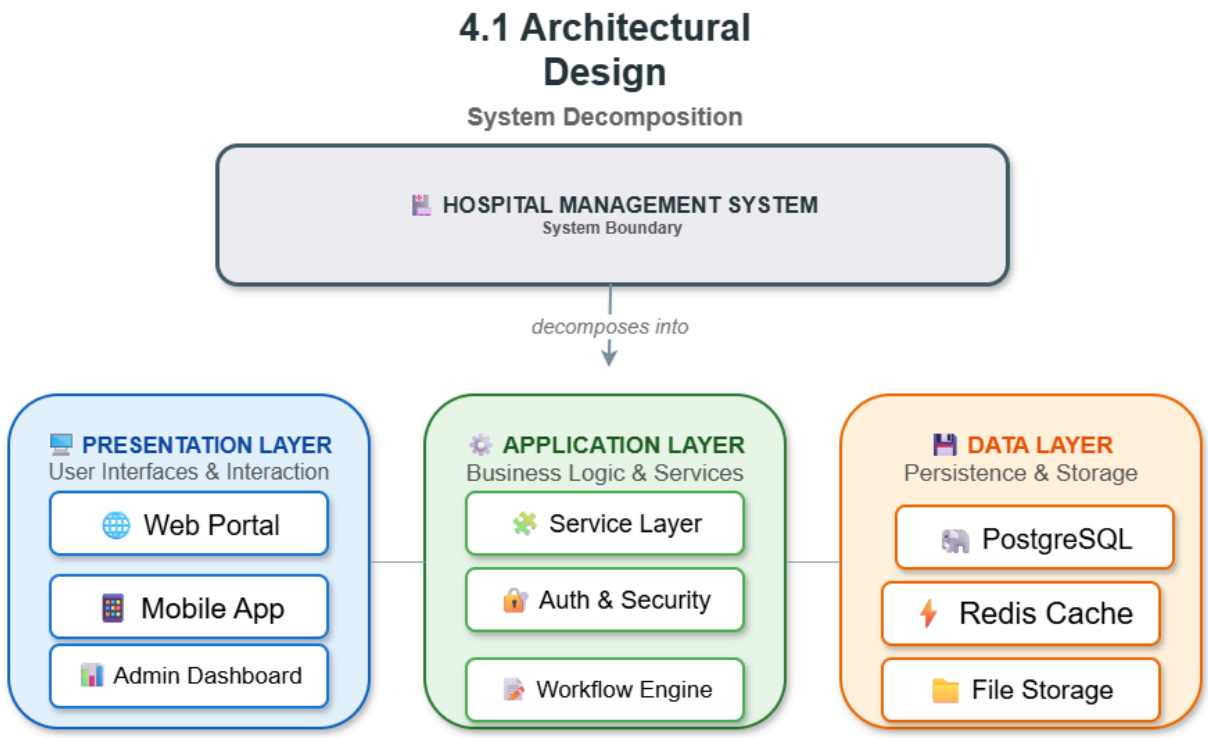
Architecture Pattern: Layered (N-Tier) Architecture

The system adopts a Layered Architecture pattern. This choice balances development speed, maintainability, and clear separation of concerns. It allows the system to evolve into a Microservices architecture in Phase 2 if required.
Layers:

1. Presentation Layer (Tier 1): Web Portal (React), Admin Dashboard (Angular). Handles user interaction.

2. Application Layer (Tier 2): API Gateway (Kong/Spring Cloud Gateway) and Business Logic Services (Spring Boot). Handles routing, security, and core business rules.
3. Data Layer (Tier 3): PostgreSQL (Relational Data), Redis (Cache), MongoDB (Logs/Documents), AWS S3 (Files). Handles persistence.

4.1 Architectural Design

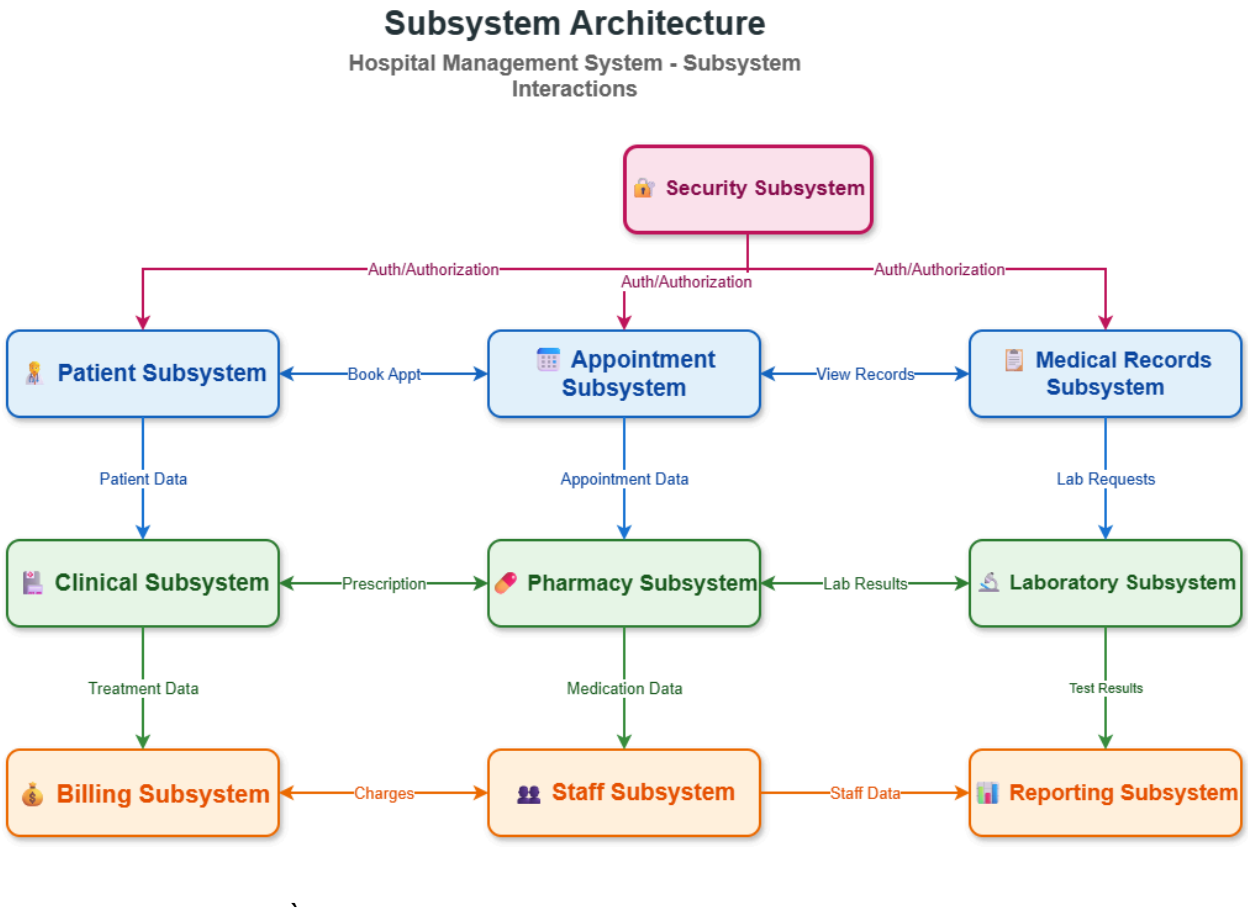


Subsystem Decomposition

Subsystem	Responsibility	Key Components
User Interface	Handle user interactions	Web Portal, Mobile App, Admin Panel
Patient Mgmt	Patient lifecycle	Registration, Profile, History
Appointment	Scheduling	Booking Engine, Calendar, Notifications

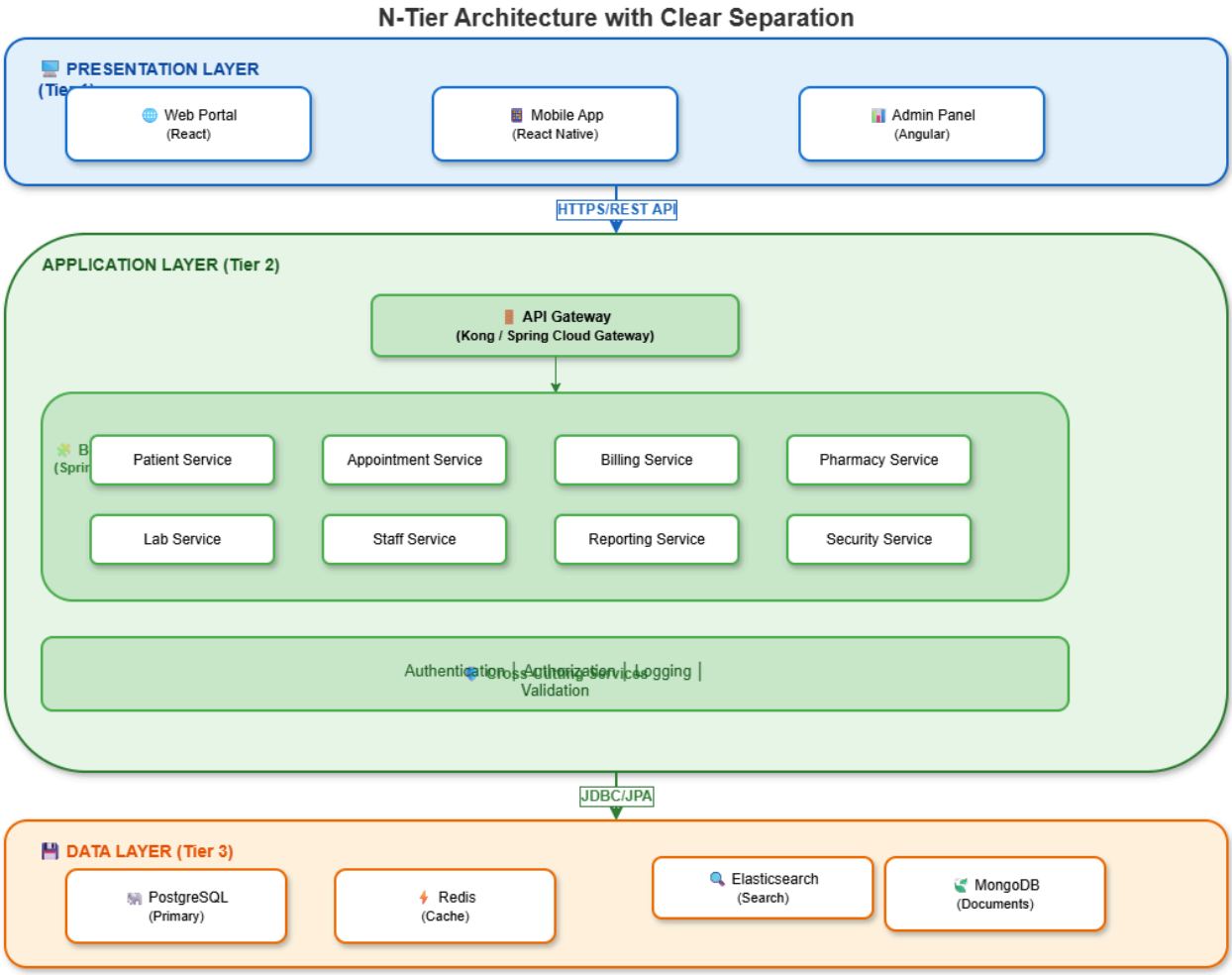
Medical Records	EMR Storage	Record Manager, Document Storage
Clinical	Clinical Ops	Prescription, Lab Orders, Diagnosis
Billing	Financials	Invoice Generator, Payment Processor
Security	Access Control	Auth, Authorization, Audit Logs

Subsystem Interaction Diagram:



Architecture Pattern: LAYERED ARCHITECTURE
The system uses a Layered (N-Tier) Architecture pattern with the following layers:

Layered Architecture Pattern



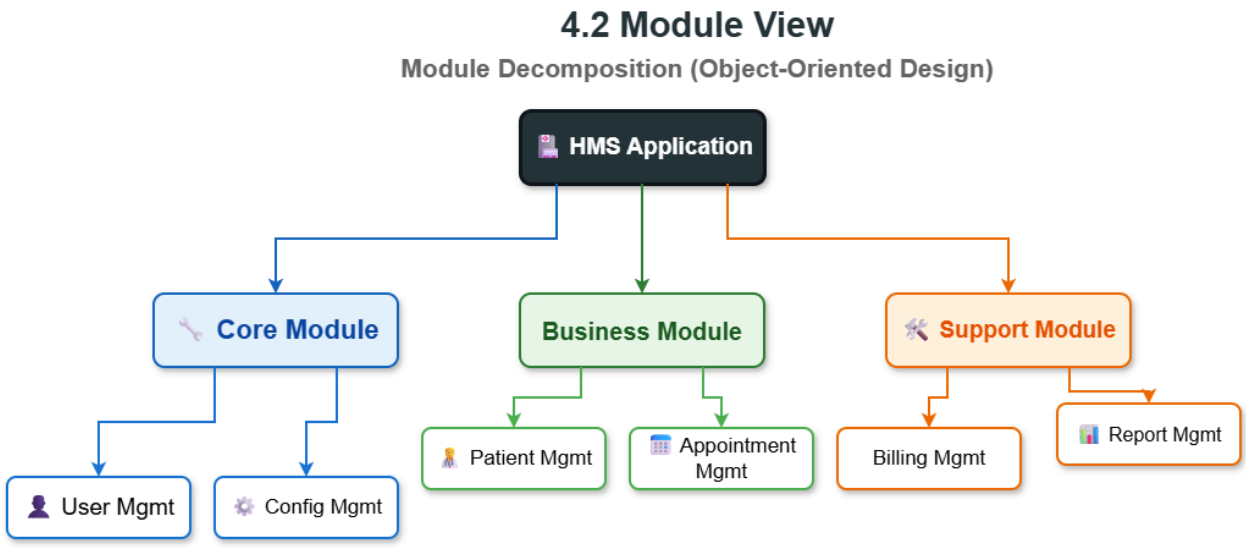
Why Layered Architecture?

Advantage	Description
Separation of Concerns	Each layer has distinct responsibility
Maintainability	Easy to modify one layer without affecting others

Testability	Layers can be tested independently
Scalability	Individual layers can be scaled
Reusability	Business logic can be reused across different UIs
Standardization	Well-understood pattern, easier onboarding

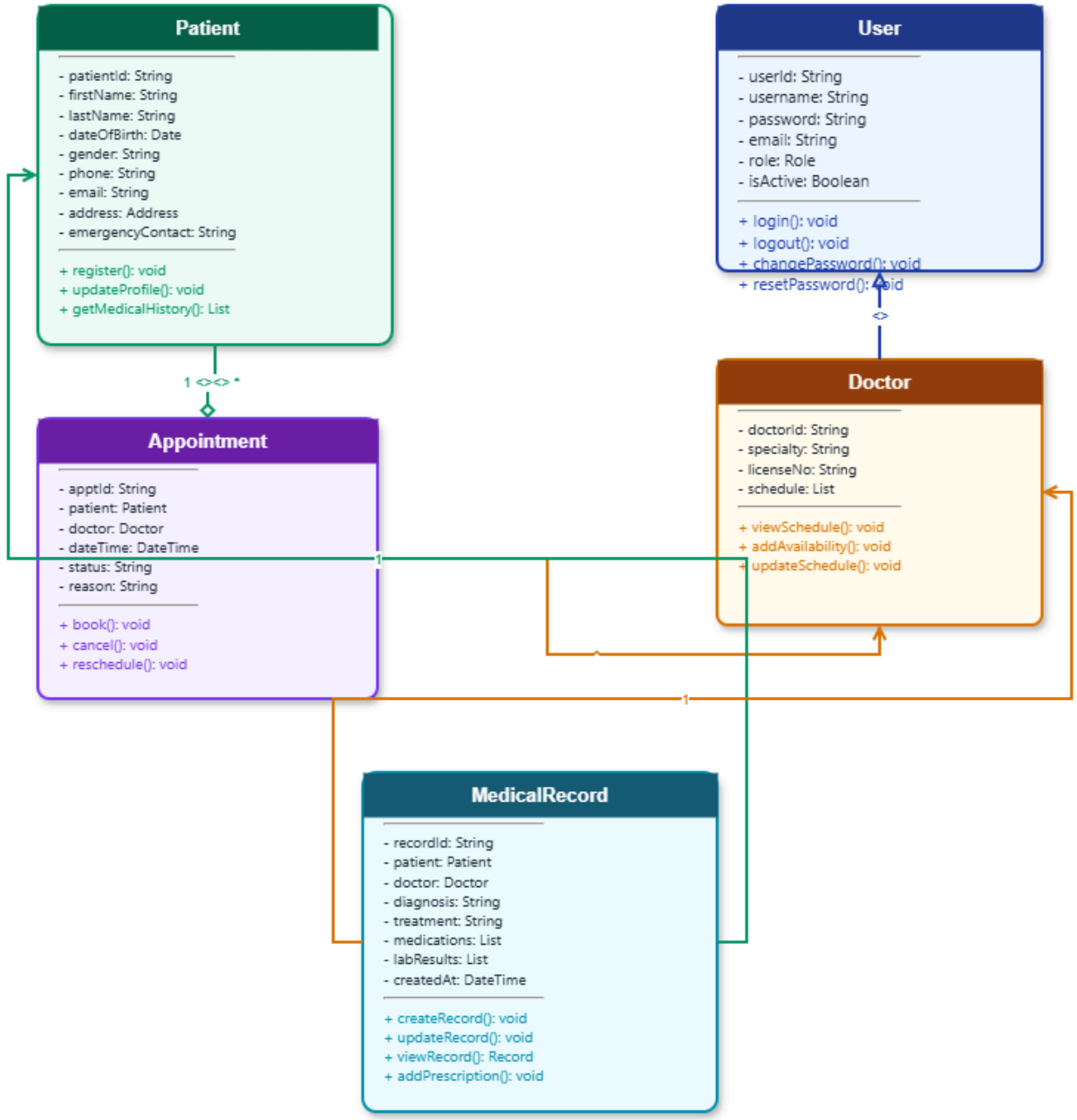
4.2 Module View

Module Decomposition:
The system is decomposed into the following modules using object-oriented Design:



Class Diagram (Key Entities)
The system follows an Object-Oriented design. Key entities include `Patient`, `Doctor`, `Appointment`, `MedicalRecord`, and `User`.

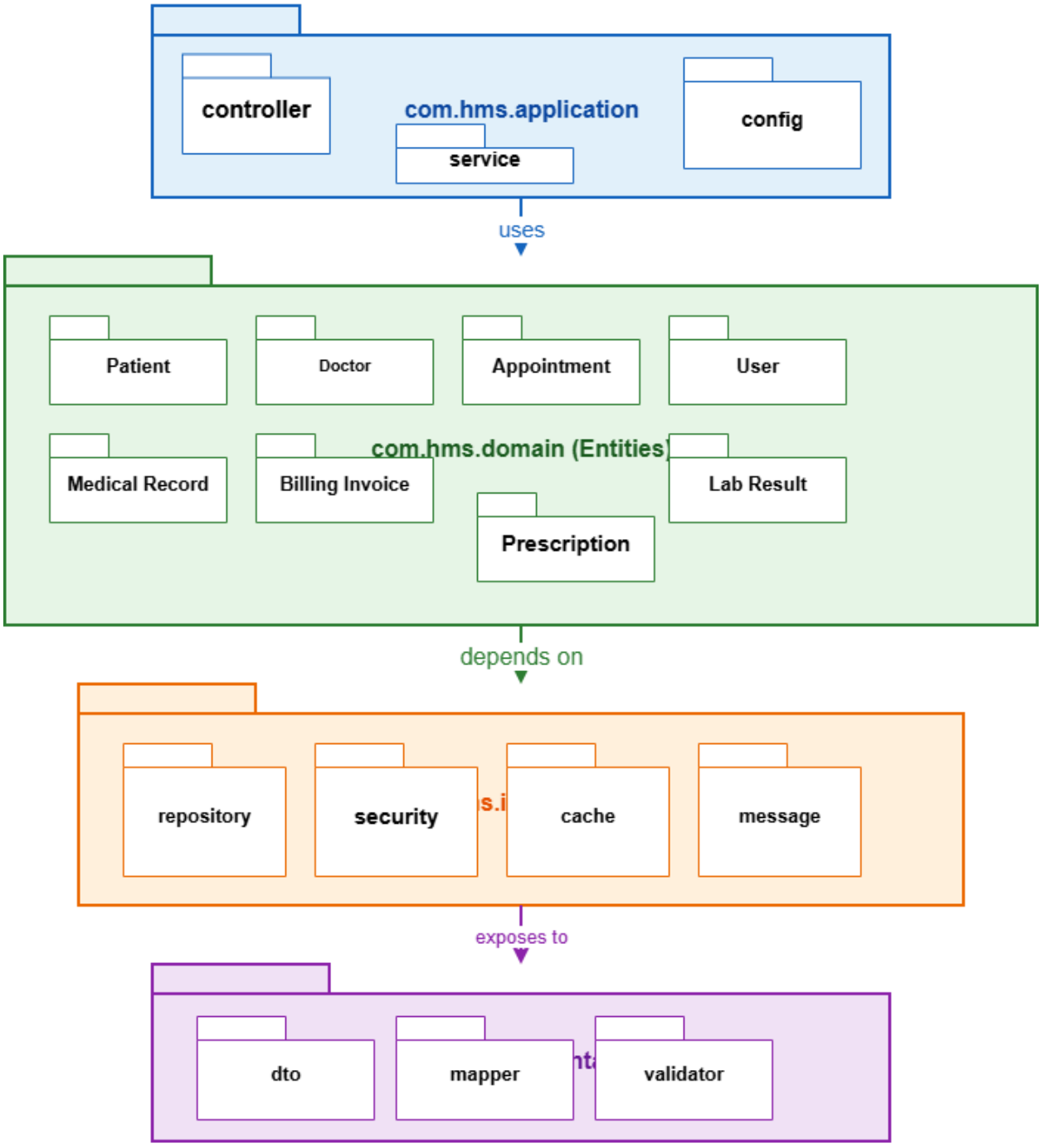
📘 UML Class Diagram: Key HMS Entities



Package Diagram:

Package Diagram

Hospital Management System

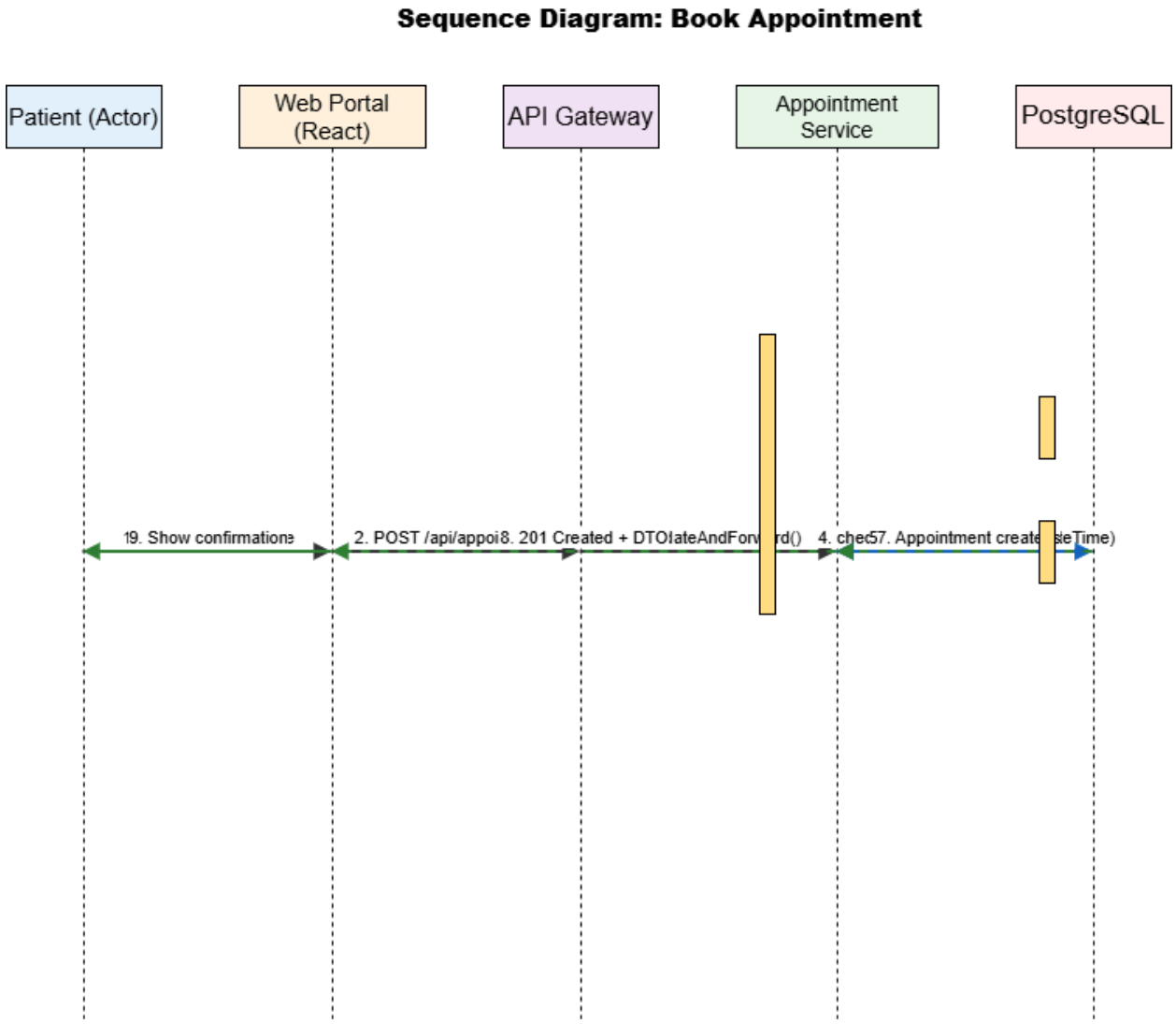


📌 **Package Dependencies:**

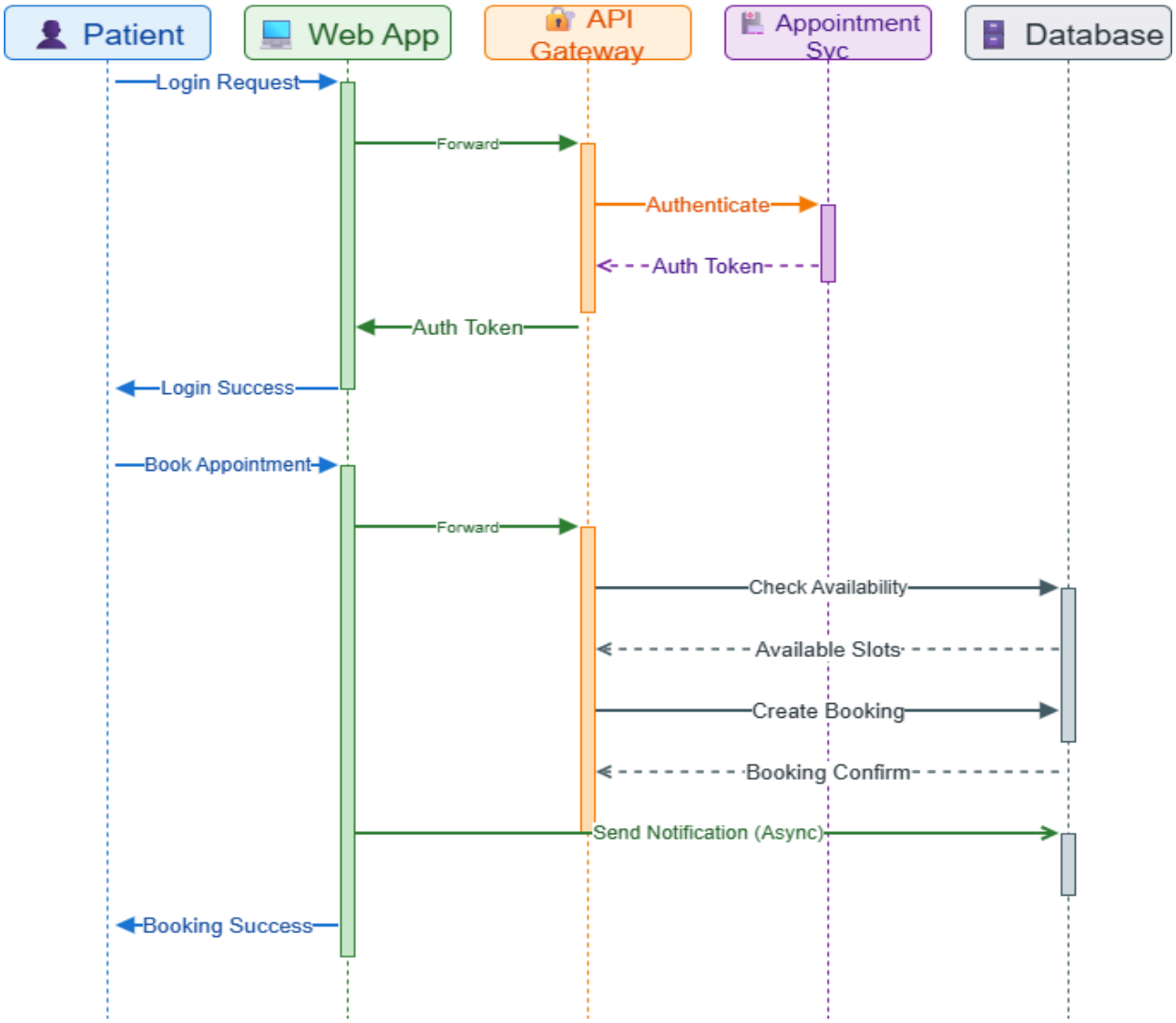
- **Application** orchestrates flows using Domain & Presentation
- **Domain** contains core entities & business rules
- **Infrastructure** provides DB, cache, security, & messaging
- **Presentation** handles DTOs, mapping, & validation

4.3 Component and Connector View

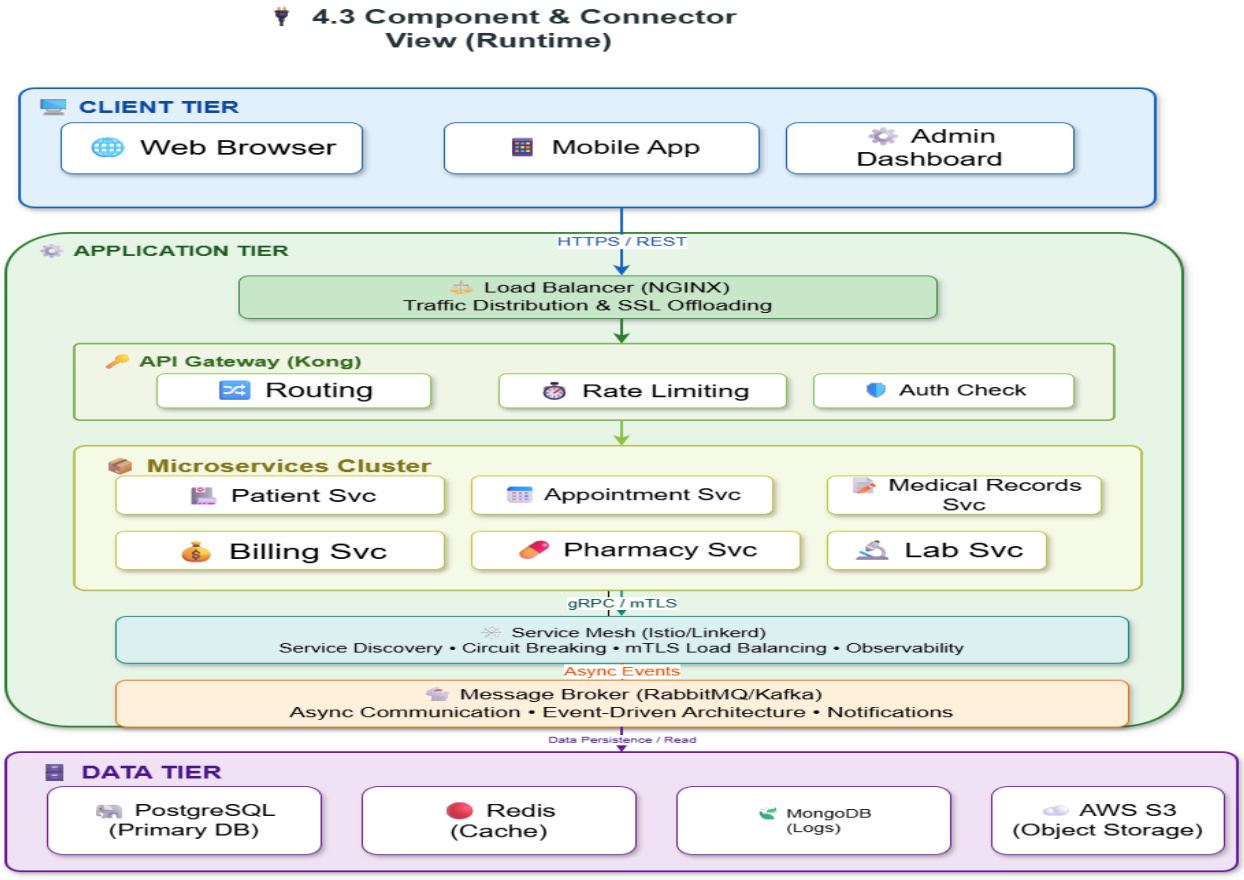
Sequence Diagram: Patient Registration



Sequence Diagram: Patient Appointment Booking



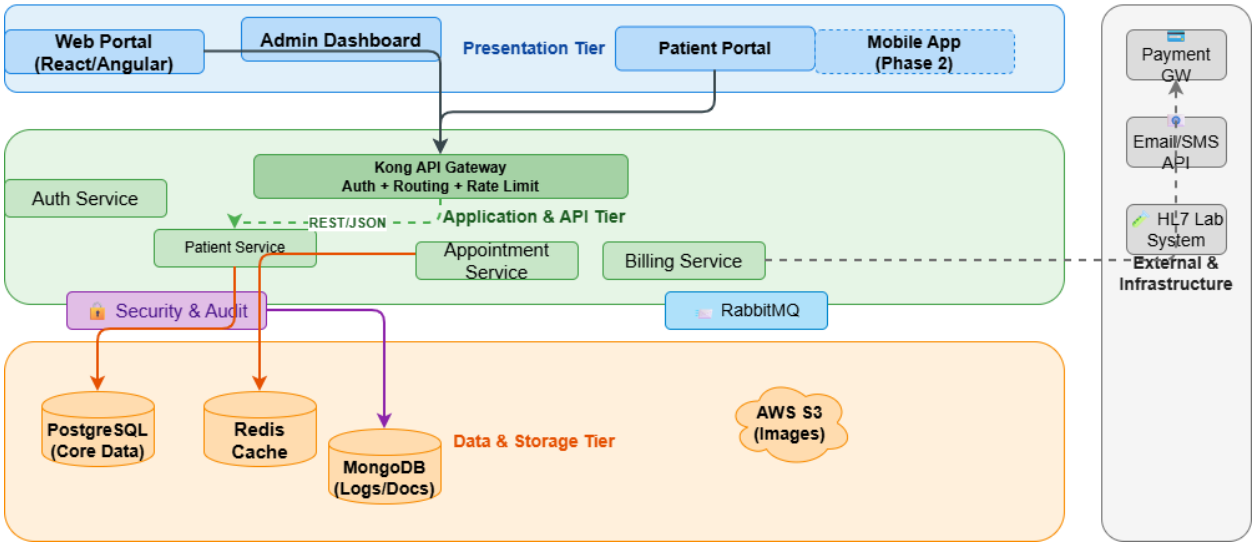
:



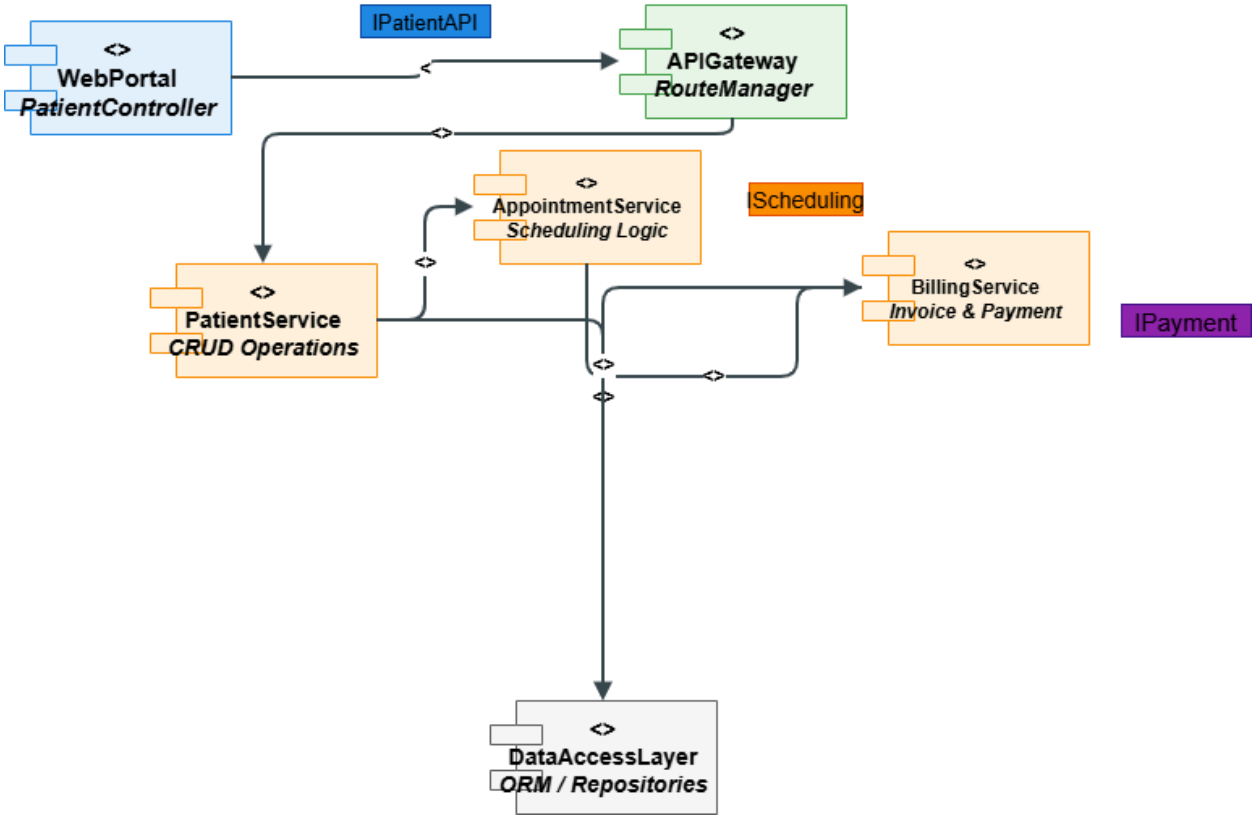
4.4 Allocation View

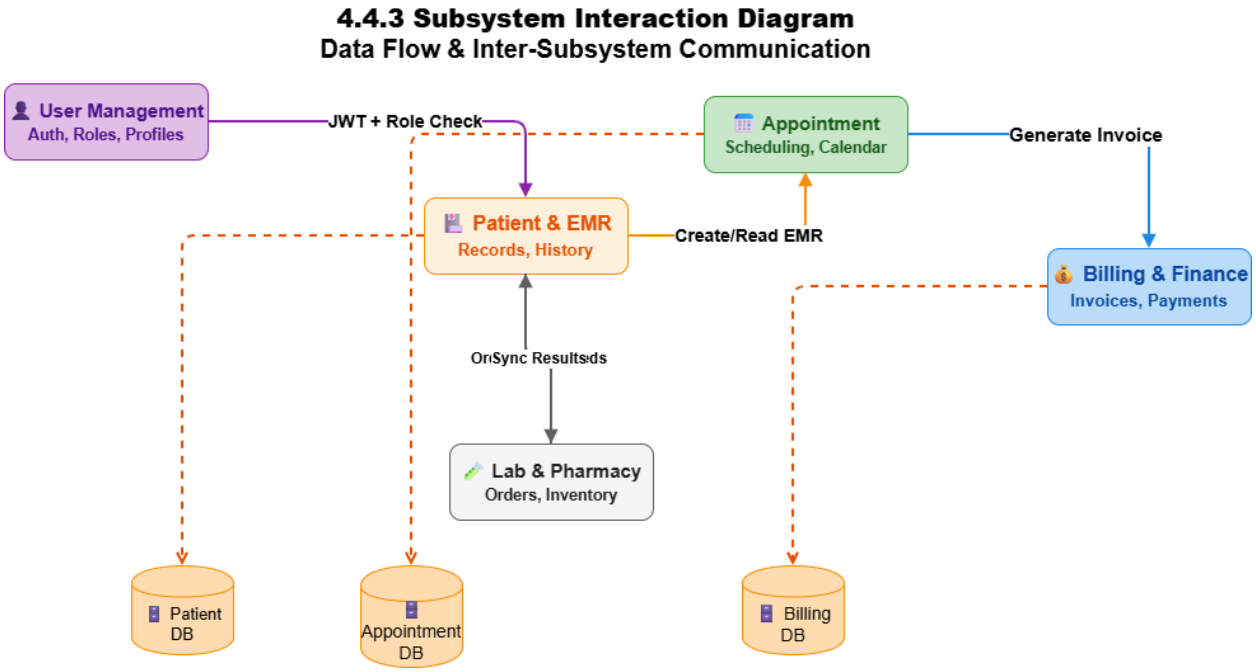
4.4.1 High-Level Architecture Diagram

3-Tier Layered + Microservices-Ready



4.4.2 UML Component Diagram
Interfaces, Dependencies & Data Access

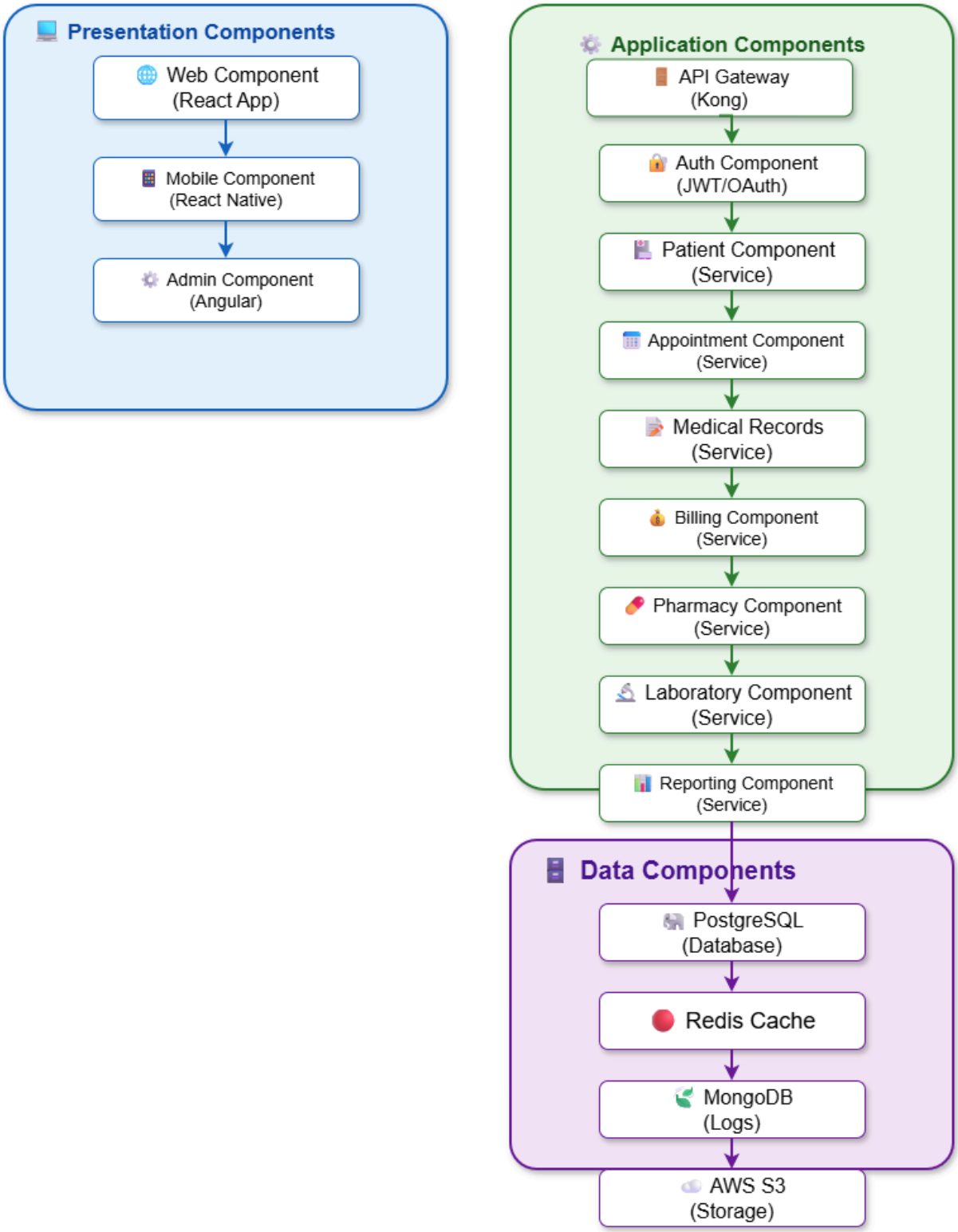


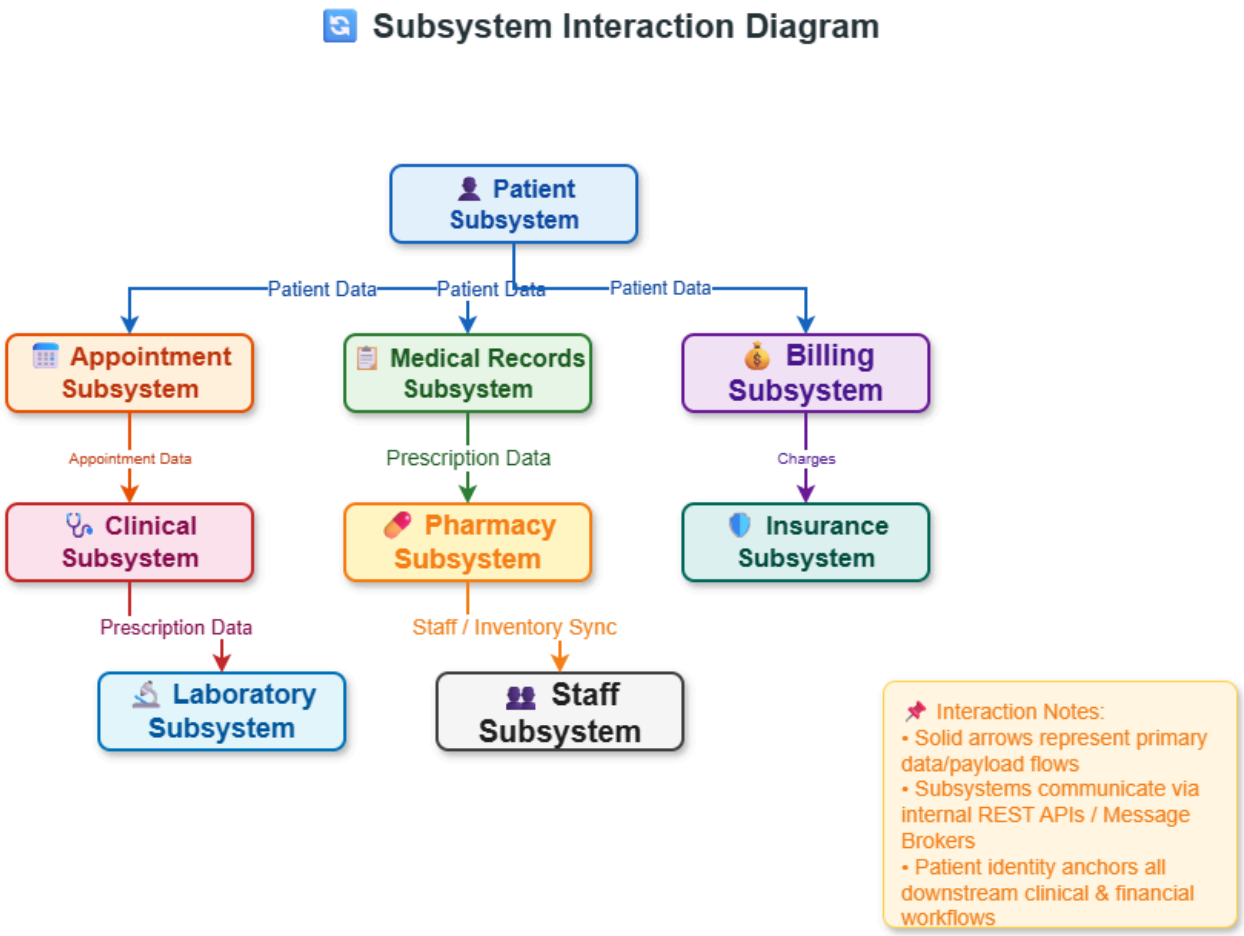


Deployment Diagram

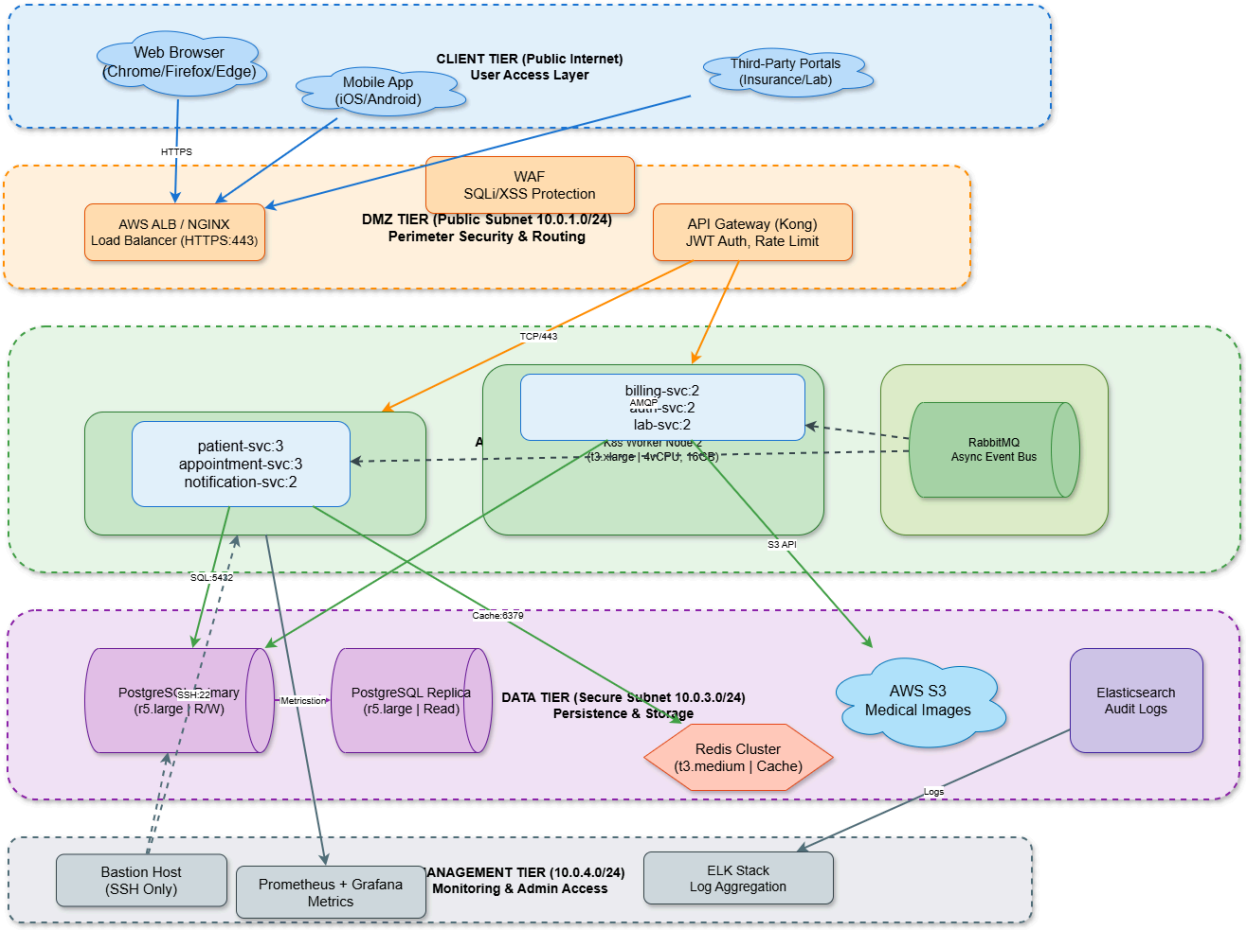
Component Diagram:

HMS System Components

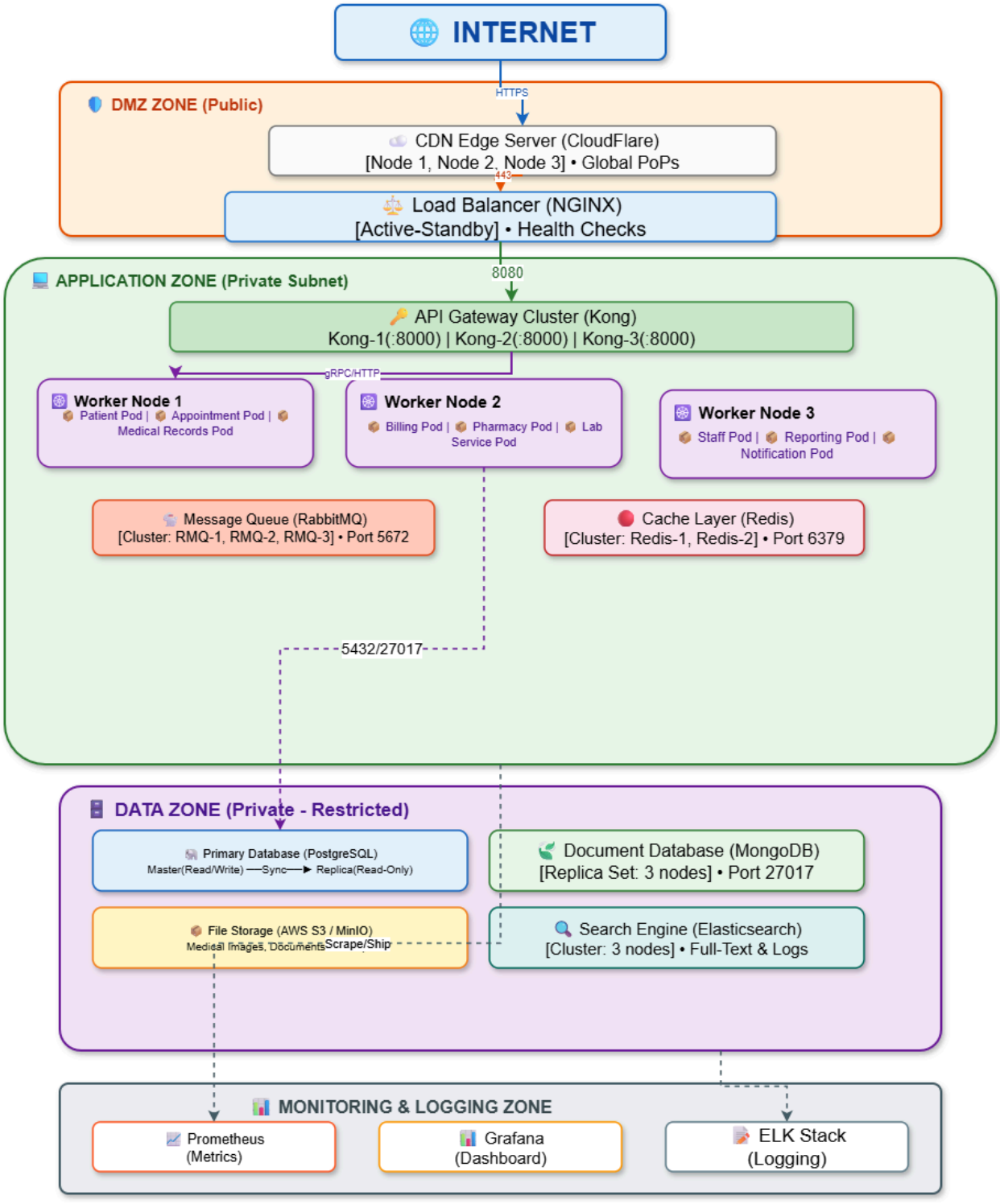




Deployment Diagram:


Web Browser
(Chrome/Firefox/Edge)
Mobile App
(iOS/Android)
CLIENT TIER (Public Internet)
User Access Layer
Third-Party Portals
(Insurance/Lab)
HTTPS
WAF
SQLi/XSS Protection
AWS ALB / NGINX
Load Balancer (HTTPS:443)
DMZ TIER (Public Subnet 10.0.1.0/24)
Perimeter Security & Routing
API Gateway (Kong)
JWT Auth, Rate Limit
TCP/443
billing-svc:2
AMQP
lab-svc:2
patient-svc:3
appointment-svc:3
notification-svc:2
RabbitMQ
Async Event Bus
S3 API
SQL:5432
Cache:6379
PostgreSQL Primary
(r5.large | R/W)
Metrics
PostgreSQL Replica
(r5.large | Read)
DATA TIER (Secure Subnet 10.0.3.0/24)
Persistence & Storage
AWS S3
Medical Images
Elasticsearch
Audit Logs
Redis Cluster
(t3.medium | Cache)
Logs
SSH:22
Bastion Host
(SSH Only)
Prometheus + Grafana
Metrics
ELK Stack
Log Aggregation

4.5 Physical Deployment View



4.5 Design Rationale

Decision	Alternatives Considered	Trade-offs	Justification
Architectural Pattern	Monolithic vs. Pure Microservices vs. Layered	Monolith: Hard to scale. Pure Microservices: High operational complexity.	Layered with Microservices Readiness. Provides clear separation of concerns (Layered) while allowing individual business modules (Appointment, Billing) to scale independently if needed.
Communication Protocol	SOAP vs. gRPC vs. REST	SOAP: Heavy XML overhead. gRPC: High performance but steep learning curve.	RESTful API. Industry standard, easy to consume by Web/Mobile clients, and supported by extensive tooling (Swagger/OpenAPI).
Database Choice	MySQL vs. MongoDB vs. PostgreSQL	MySQL: Good but weaker JSON support. MongoDB: Lacks ACID for billing.	PostgreSQL. Offers strong ACID compliance for financial/medical data, excellent JSONB support for flexible medical records, and robust indexing.
Caching Strategy	No Cache vs. Memcached vs. Redis	Memcached: Simple but limited data structures.	Redis. Supports complex data structures (lists, sets) for session management and appointment slots, plus pub/sub for real-time notifications.
Security Model	Session-Based vs. JWT	Session: Requires server-side state, hard to scale horizontally.	JWT (JSON Web Tokens). Stateless authentication allows any microservice instance to validate users without querying a central session store, enhancing scalability.

Environment Description:

Environment	Purpose	Configuration
Development	Developer workstations	Docker Desktop, Local Kubernetes (Minikube)
Testing/QA	Integration & testing	Kubernetes cluster (4 nodes), Test databases
Staging	Pre-production validation	Production-like environment, sanitized data
Production	Live system	High-availability cluster, disaster recovery

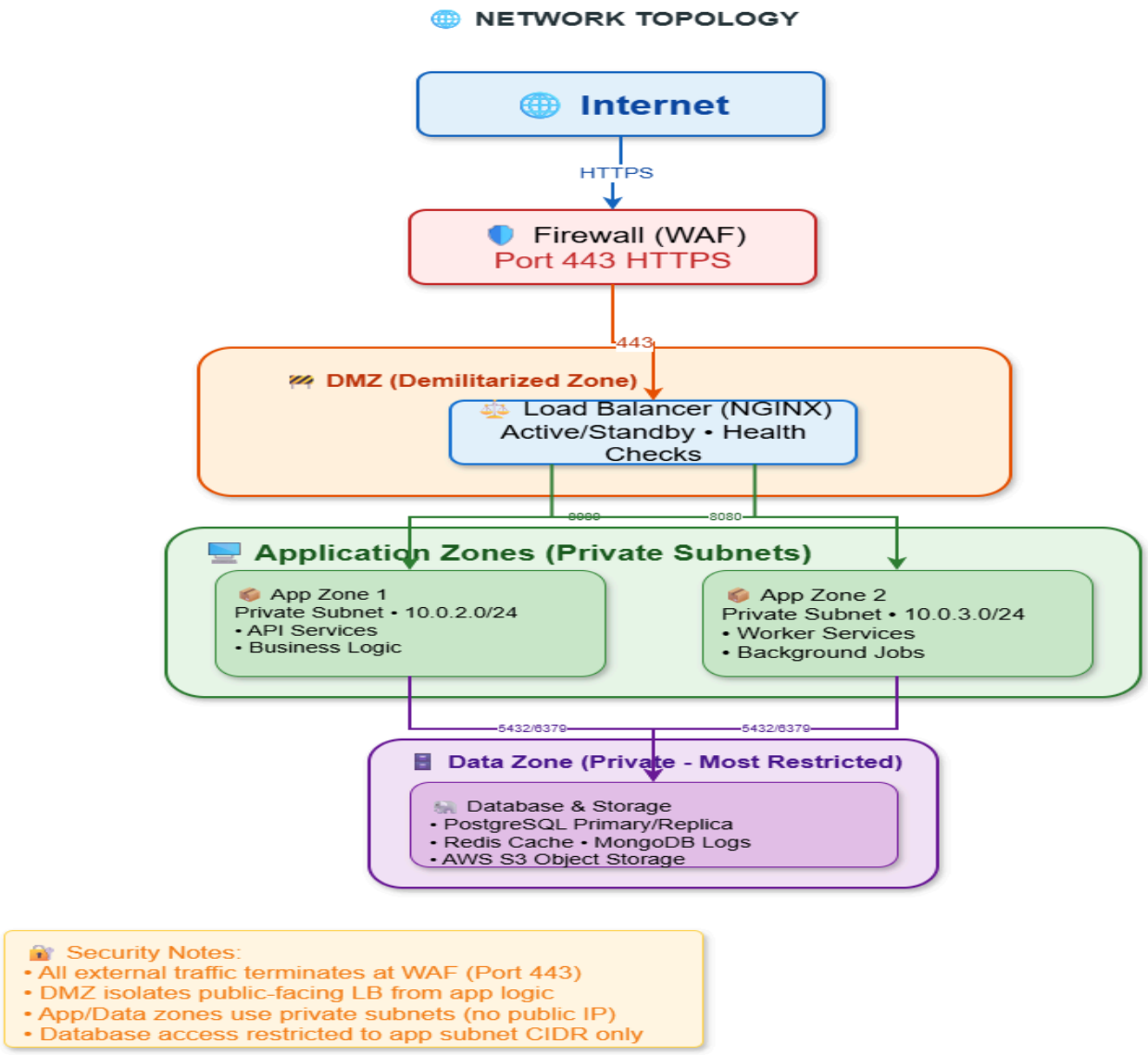
Hardware Requirements:

Component	Development	Testing	Staging	Production
Application Servers	1 node (4 CPU, 8GB RAM)	2 nodes (4 CPU, 8GB RAM)	3 nodes (8 CPU, 16GB RAM)	6 nodes (16 CPU, 32GB RAM)
Database Server	1 instance (4 CPU, 8GB RAM)	1 instance (8 CPU, 16GB RAM)	2 instances (8 CPU, 16GB RAM)	3 instances (16 CPU, 64GB RAM)
Cache Server	1 instance (2 CPU, 4GB RAM)	1 instance (2 CPU, 4GB RAM)	2 instances (4 CPU, 8GB RAM)	3 instances (8 CPU, 16GB RAM)
Storage	100 GB SSD	500 GB SSD	1 TB SSD	5 TB SSD + S3
Network	1 Gbps	1 Gbps	10 Gbps	10 Gbps red

Software Requirements:

Category	Software	Version
Container Runtime	Docker	20.10+
Orchestration	Kubernetes	1.24+
Database	PostgreSQL	14.5+
Cache	Redis	7.0+
Message Queue	RabbitMQ	3.11+
Web Server	NGINX	1.22+
API Gateway	Kong	3.0+
Monitoring	Prometheus + Grafana	Latest
Logging	Operating System	Ubuntu Server
CI/CD	Jenkins / GitHub Actions	Latest

Network Configuration:



Network Segments:

Segment	CIDR	Purpose	Access
DMZ	10.0.1.0/24	Public-facing services	Internet → DMZ
Application	10.0.2.0/24	Microservices	DMZ → App, App → App
Data	10.0.3.0/24	Databases, Storage	App → Data only
Management	10.0.4.0/24	Admin, Monit	

Architecture Pattern Selection:
Why Layered Architecture?

CCriterion	Layered Architecture	Microservices	Monolithic	Decision
Complexity	Moderate	High	Low	✓ Layered
Team Size	Medium (5-10)	Large (10+)	Small (<5)	✓ Layered
Deployment	Simple	Complex	Simple	✓ Layered
Scalability	Good	Excellent	Poor	✓ Layered
Maintainability	Good	Excellent	Poor	✓ Layered
Time to Market	Fast	Slow	Fast	✓ Layered
Team Expertise	Standard	Advanced	Standard	✓ Layered

4.6 Design Rationale

Architecture Pattern Selection:

Criterion	Layered Architecture	Microservices	Monolithic	Decision
Complexity	Moderate	High	Low	✓ Layered
Team Size	Medium (5-10)	Large (10+)	Small (<5)	✓ Layered
Deployment	Simple	Complex	Simple	✓ Layered
Scalability	Good	Excellent	Poor	✓ Layered
Maintainability	Good	Excellent	Poor	✓ Layered
Time to Market	Fast	Slow	Fast	✓ Layered
Team Expertise	Standard	Advanced	Standard	✓ Layered

Rationale:

- Layered architecture provides the best balance for our hospital management system
- Clear separation of concerns improves maintainability
- Easier to test individual layers independently
- Can evolve to microservices if needed in future
- Fits well with team size and expertise
- Faster development and deployment cycles

Technology Stack Decisions:
Backend Framework: Spring Boot

Factor	Spring Boot	Node.js	Django	Decision
Performance	High	High	Medium	✓
Enterprise Features	Excellent	Good	Good	✓
Security	Excellent	Good	Good	✓
Team Expertise	Strong	Medium	Low	✓
Ecosystem	Rich	Rich	Rich	✓
Type Safety	Strong (Java)	Weak (JS)	Medium (Python)	✓

CHAPTER 5. DATA DESIGN

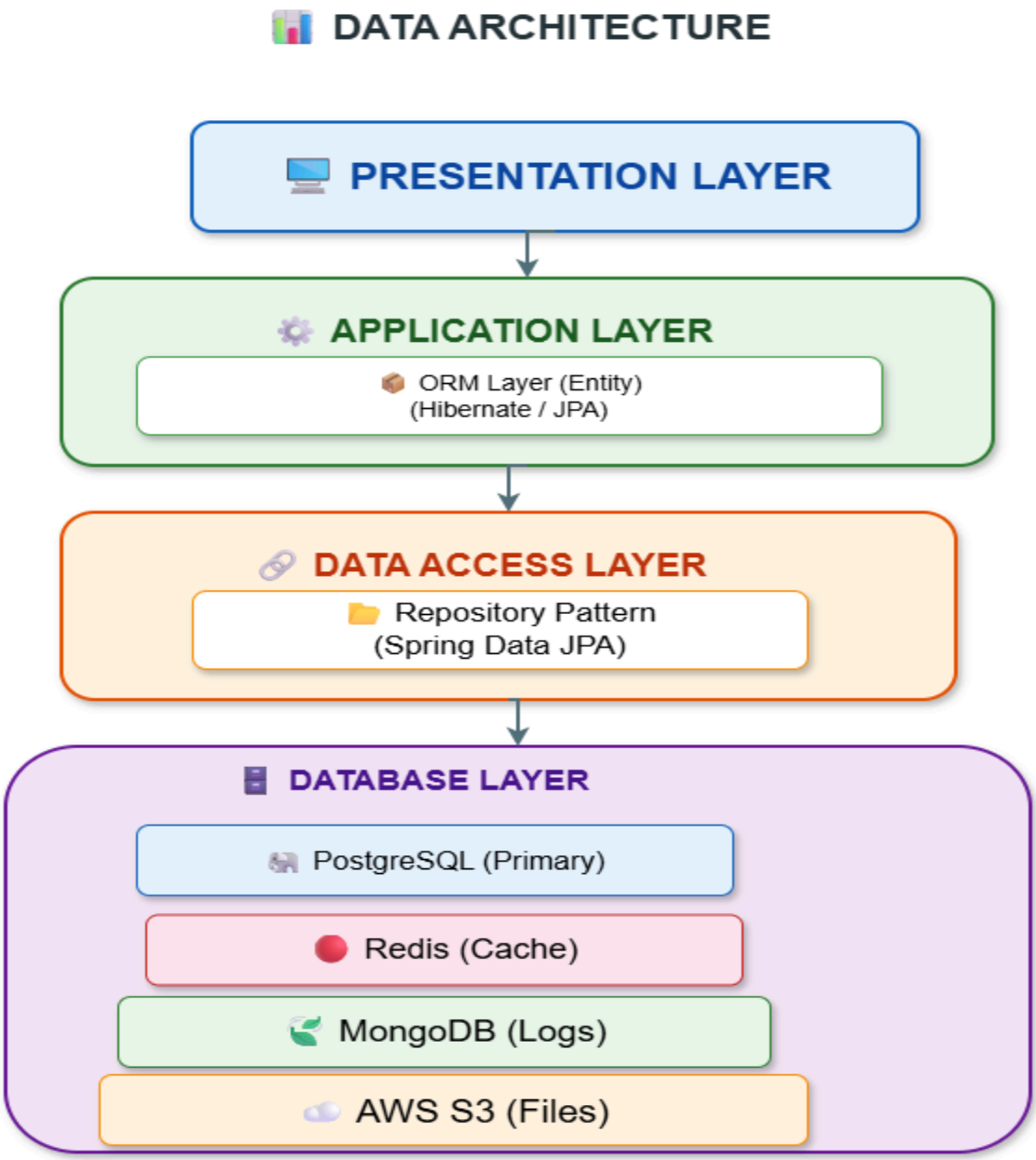
5.1 Data Description

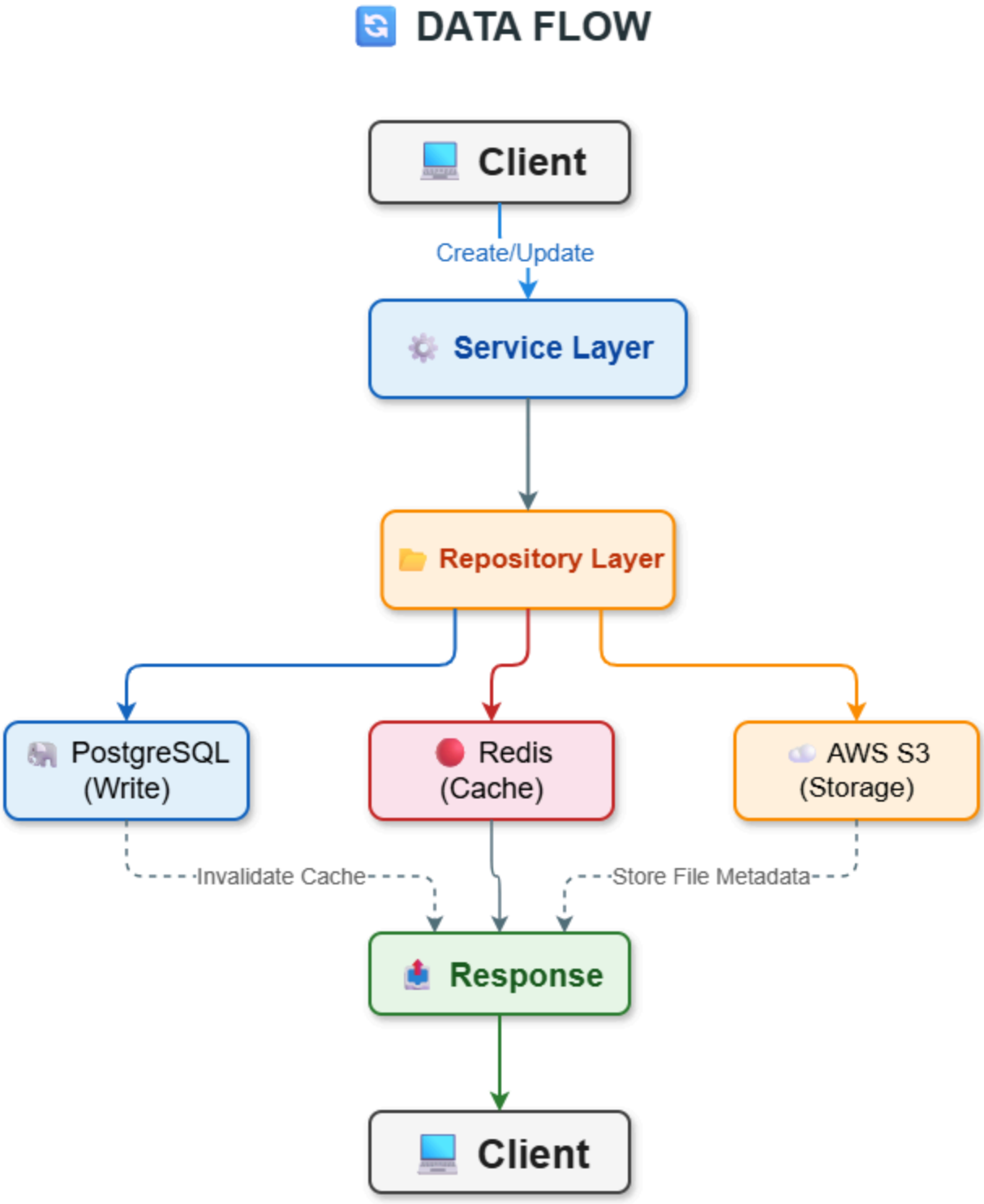
Information Domain Transformation:
The Hospital Management System transforms real-world healthcare entities into structured data:

Real-World Entity	→	Data Structure
Patient	→	Patient Table + Medical Records
Doctor	→	Doctor Table + Schedule
Appointment	→	Appointment Table + Calendar
Medication	→	Drug Table + Prescription Table
Laboratory Test	→	Test Table + Results Table
Billing Invoice	→	Invoice Table + Payment Table
Medical Image	→	S3 Storage + Metadata Table

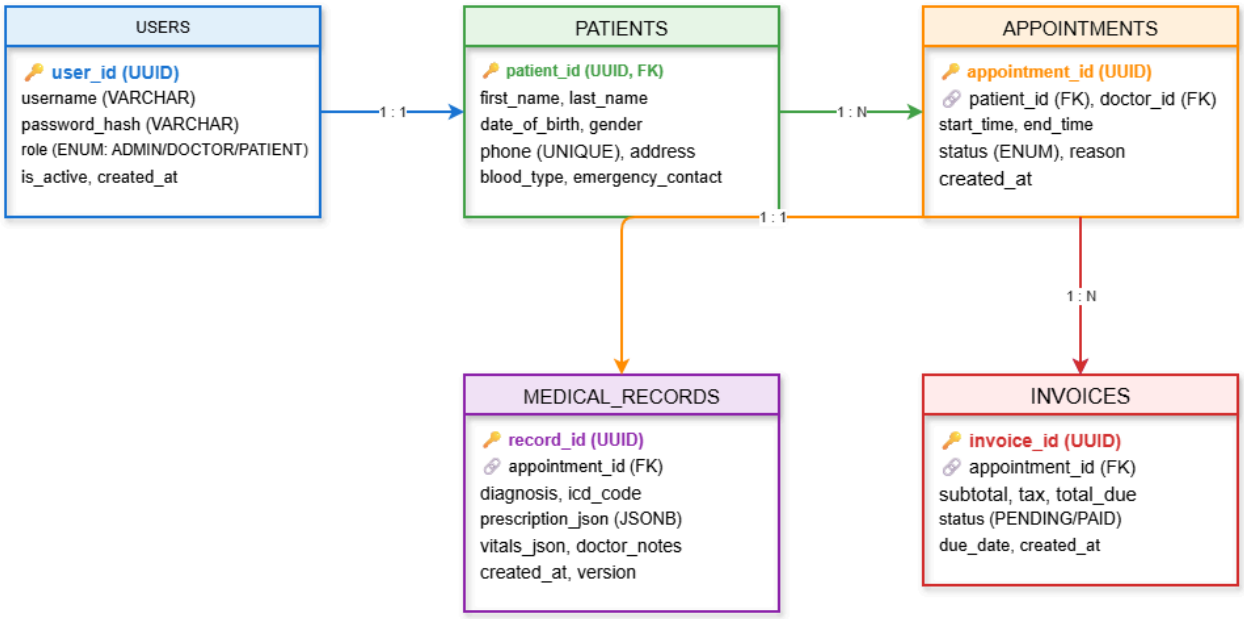
Data Type	Storage	Rationale
Structured Data	PostgreSQL (Relational)	ACID compliance, complex queries
Session Data	Redis (In-memory)	Fast access, automatic expiry
Documents/Logs	MongoDB (Document)	Flexible schema, hierarchical data
Medical Images	AWS S3 (Object Storage)	Scalable, cost-effective
Search Index	Elasticsearch	Full-text search, analytics
Cache	Redis	Reduce database load

Data Storage Strategy:Data Organization:





5.2 Data Dictionary

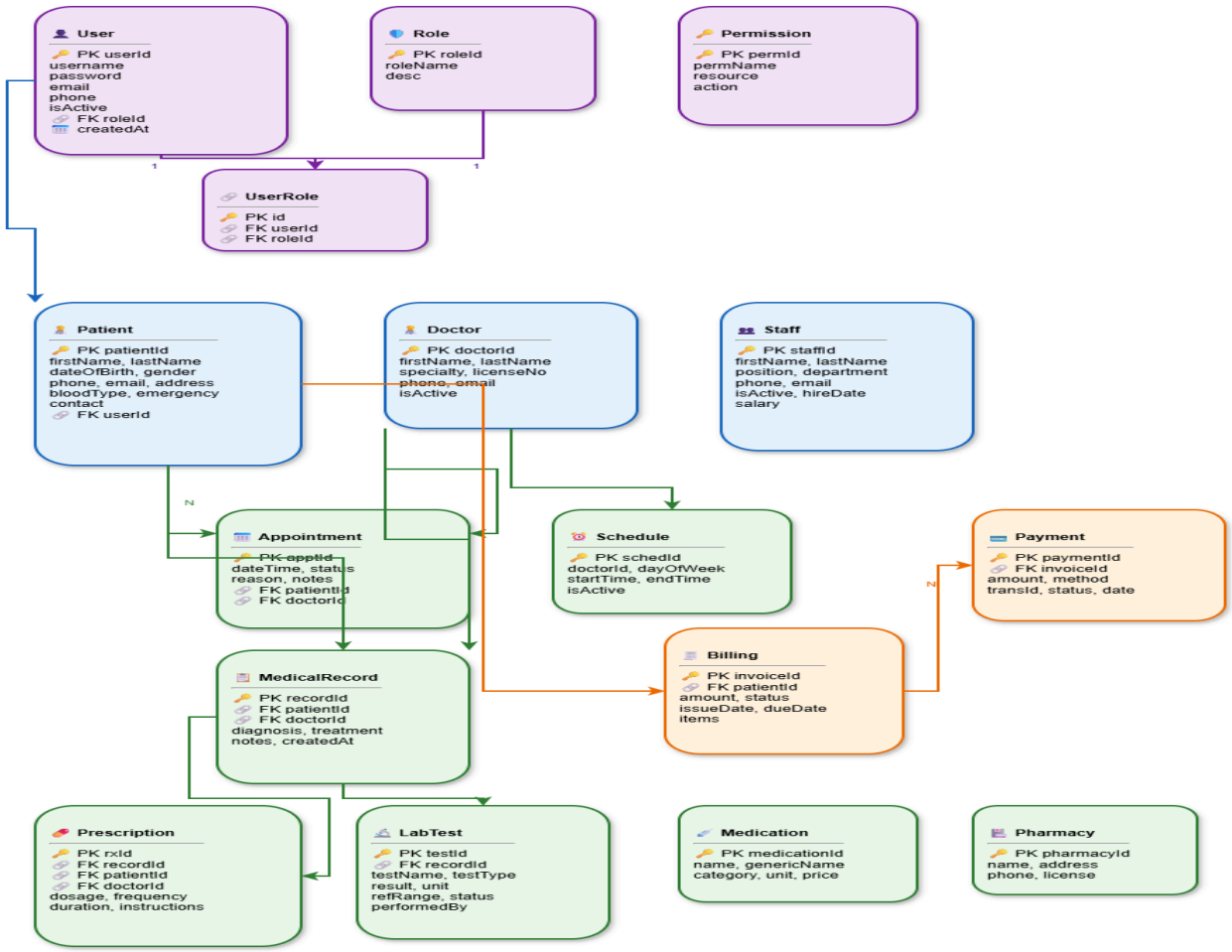


Database Schema:

Entity-Relationship Diagram (ERD):

5.2 Data Dictionary

Database Schema & Entity-Relationship Diagram (ERD)



Detailed Table Definitions:

1. User Table

Attribute	Type	Description	Constraints
user_id	UUID	Unique Identifier	PK, NOT NULL
username	VARCHAR(50)	Login Username	UNIQUE, NOT NULL
password_hash	VARCHAR(255)	BCrypt Hashed Password	NOT NULL
email	VARCHAR(100)	Contact Email	UNIQUE, NOT NULL
role	ENUM	Role Type	ADMIN, DOCTOR, NURSE, PATIENT, STAFF
is_active	BOOLEAN	Account Status	DEFAULT TRUE
created_at	TIMESTAMP	Record Creation	DEFAULT NOW()

2. Table: `patients`

Attribute	Type	Description	Constraints
patient_id	UUID	Unique Identifier	PK, FK → users.user_id
first_name	VARCHAR(100)	First Name	NOT NULL
last_name	VARCHAR(100)	Last Name	NOT NULL
date_of_birth	DATE	Date of Birth	NOT NULL
gender	VARCHAR(10)	Gender	M/F/O
phone	VARCHAR(20)	Primary Phone	UNIQUE, NOT NULL
address	TEXT	Residential Address	
emergency_contact	VARCHAR(200)	Emergency Contact Details	
blood_type	VARCHAR(5)	Blood Group	A+, O-, etc.

3. Table: `doctors`

Attribute	Type	Description	Constraints
doctor_id	UUID	Unique Identifier	PK, FK → users.user_id
license_number	VARCHAR(50)	Medical License ID	UNIQUE, NOT NULL
specialty	VARCHAR(100)	Medical Specialty	Cardiology, Neurology, etc.
consultation_fee	DECIMAL(10,2)	Standard Fee	>= 0
availability_json	JSONB	Weekly Schedule Template	

4. Table: `appointments`

Attribute	Type	Description	Constraints
appointment_id	UUID	Unique Identifier	PK, NOT NULL
patient_id	UUID	Foreign Key	FK → patients.patient_id
doctor_id	UUID	Foreign Key	FK → doctors.doctor_id
start_time	TIMESTAMP	Start Date & Time	NOT NULL
end_time	TIMESTAMP	End Date & Time	NOT NULL

status	ENUM	Status	SCHEDULED, COMPLETED, CANCELLED, NO_SHOW
reason	TEXT	Reason for Visit	
created_at	TIMESTAMP	Booking Time	DEFAULT NOW()

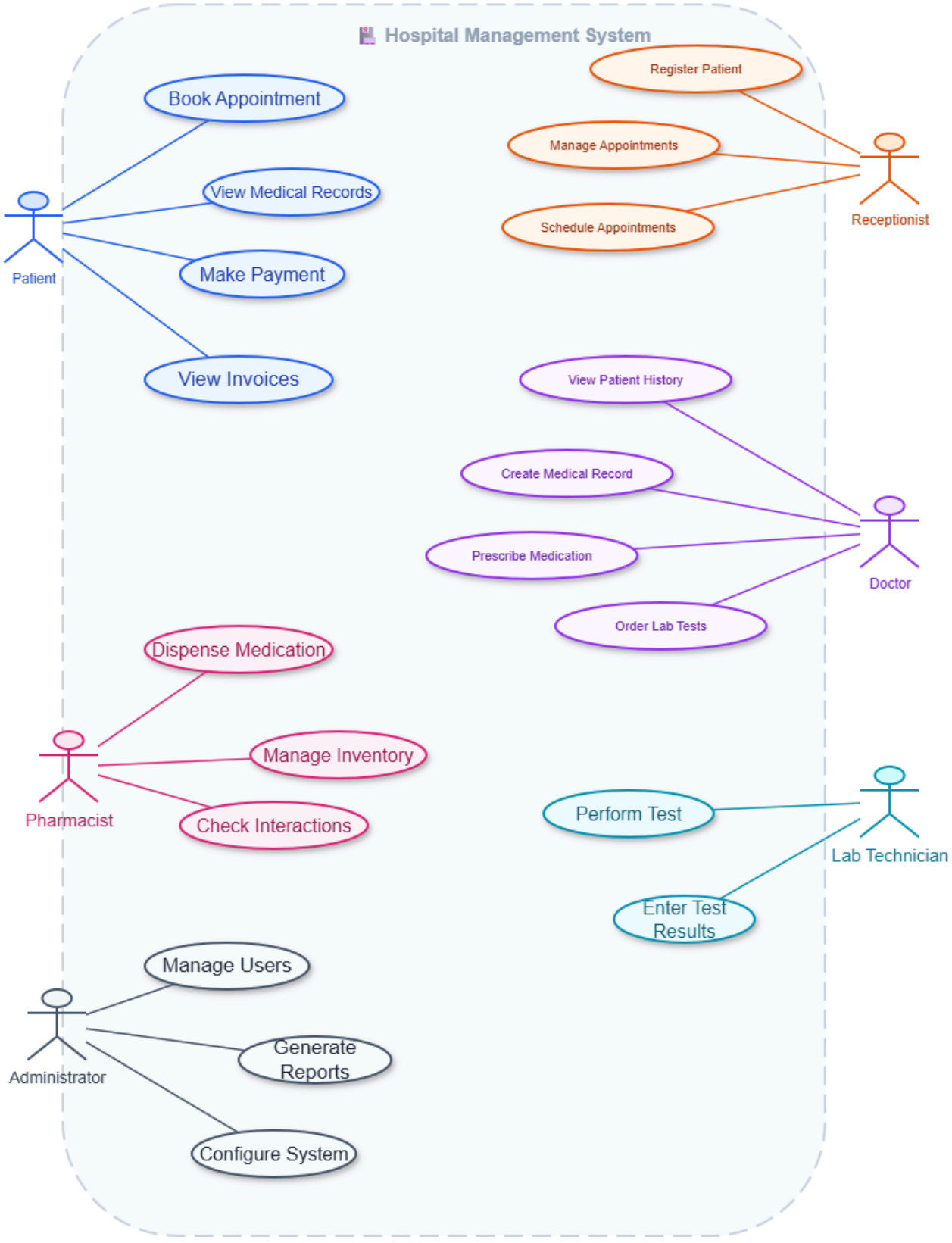
5. Table:

Attribute	Type	Constraints	Constraints
record_id	UUID	Unique Identifier	PK, NOT NULL
appointment_id	UUID	Linked Appointment	FK → appointments.appointment_id
diagnosis	TEXT	Diagnosis Notes	
icd_code	VARCHAR(20)	ICD-10 Code	Optional
prescription_json	JSONB	Medications & Dosage	Flexible structure
lab_orders_json	JSONB	Ordered Tests	
vitals_json	JSONB	BP, Heart Rate, Temp	
doctor_notes	TEXT	Detailed Clinical Notes	
created_at	TIMESTAMP	Record Creation	DEFAULT NOW()

CHAPTER 6. COMPONENT DESIGN

6.1 Use Case Diagram

6.1 HMS Use Case Diagram



Detailed Use Case Descriptions:
Use Case 1: Register Patient

Field	Description
Use Case ID	UC-001
Use Case Name	Register Patient
Primary Actor	Receptionist
Secondary Actor	Patient
Description	Register a new patient in the hospital system
Preconditions	Receptionist is logged in
Postconditions	Patient record created, patient ID generated
Trigger	New patient arrives at hospital

Main Flow:

1. Receptionist selects "Register New Patient"
2. System displays patient registration form
3. Receptionist enters patient demographics (name, DOB, gender, contact info)
4. Receptionist enters emergency contact details
5. Receptionist enters medical history and allergies
6. System validates all required fields
7. System checks for duplicate patient (by phone/email)
8. System creates user account
9. System generates unique patient ID
10. System saves patient record
11. System displays confirmation with patient ID
12. System prints patient registration card

Alternative Flows:

- A1: Duplicate Patient Found
    - Step 7a: System detects existing patient with same phone/email
    - Step 7b: System displays existing patient record
    - Step 7c: Receptionist verifies if same patient
    - Step 7d: If yes, update existing record; if no, continue registration
- A2: Validation Error
    - Step 6a: System detects missing or invalid data
    - Step 6b: System highlights errors
    - Step 6c: Receptionist corrects errors
    - Step 6d: Return to step 6

Exception Flows:

- E1: System Error
    - At any step: Database connection fails
    - System displays error message
    - Receptionist retries after connection restored

Business Rules:

- BR-001: Patient must have unique phone number or email
- BR-002: Date of birth cannot be in the future
- BR-003: Emergency contact is mandatory
- BR-004: Patient ID format: PAT-YYYY-XXXXX

Use Case 2: Book Appointment

Field	Description
Use Case ID	UC-002
Use Case Name	Book Appointment
Primary Actor	Receptionist/Patient
Secondary Actor	Doctor
Description	Schedule an appointment with a doctor
Preconditions	Patient is registered, Doctor is available

Postconditions	Appointment created, notifications sent
Trigger	Patient requests appointment

Main Flow:

1. Actor selects "Book Appointment"
2. System displays doctor selection (by specialty or name)
3. Actor selects doctor
4. System displays doctor's available time slots
5. Actor selects preferred date and time
6. Actor enters appointment reason/symptoms
7. System checks for conflicts
8. System creates appointment
9. System sends confirmation to patient (email/SMS)
10. System adds appointment to doctor's schedule
11. System displays appointment confirmation

Alternative Flows:

- A1: No Available Slots
    - Step 4a: No slots available for selected date
    - Step 4b: System suggests alternative dates
    - Step 4c: Actor selects alternative or cancels
- A2: Patient Books Online
    - Step 1a: Patient logs into patient portal
    - Step 1b: System shows only patient's own appointments
    - Continue from step 2

Business Rules:

- BR-010: Appointment duration: 15-60 minutes based on type
- BR-011: Appointments can be booked up to 30 days in advance
- BR-012: Cancellation allowed up to 24 hours before
- BR-013: Maximum 5 appointments per patient per day

Use Case 3: Create Medical Record

Field	Description
Use Case ID	UC-003
Use Case Name	Create Medical Record
Primary Actor	Doctor
Secondary Actor	Patient
Description	Document patient visit and diagnosis
Preconditions	Doctor is logged in, Patient has appointment
Postconditions	Medical record created, prescriptions added
Trigger	Doctor sees patient

Main Flow:

1. Doctor selects patient from appointment list
2. System displays patient's medical history
3. Doctor enters chief complaint
4. Doctor performs examination, enters vitals
5. Doctor enters diagnosis (with ICD code)
6. Doctor prescribes medications
7. Doctor orders lab tests if needed
8. Doctor enters treatment plan
9. Doctor adds notes
10. System validates required fields
11. System saves medical record
12. System updates patient history
13. System generates prescription
14. System notifies pharmacy if prescriptions issued

Alternative Flows:

- A1: Follow-up Visit
    - Step 2a: System shows previous records
    - Step 2b: Doctor reviews previous treatment
    - Continue from step 3
- A2: Emergency Visit
    - Step 1a: Doctor accesses emergency module
    - Step 1b: Quick record creation with minimal fields
    - Step 1c: Detailed information added later

Business Rules:

- BR-020: Medical record must be completed within 24 hours
- BR-021: ICD-10 codes required for diagnosis
- BR-022: All prescriptions require doctor's digital signature
- BR-023: Records cannot be deleted, only amended

Use Case 4: Process Payment

Field	Description
Use Case ID	UC-004
Use Case Name	Process Payment
Primary Actor	Receptionist/Cashier
Secondary Actor	Patient
Description	Process payment for medical services
Preconditions	Invoice generated, Patient ready to pay
Postconditions	Payment recorded, receipt generated
Trigger	Patient presents for payment

Main Flow:

1. Actor searches for patient or invoice
2. System displays outstanding invoices
3. Actor selects invoice(s) to pay

4. System shows total amount due
5. Actor selects payment method
6. Actor enters payment details
7. System processes payment (gateway integration)
8. System updates invoice status to "Paid"
9. System creates payment record
10. System generates receipt
11. System sends receipt to patient (email/SMS)
12. System prints receipt

Alternative Flows:

- A1: Partial Payment
    - Step 5a: Patient pays partial amount
    - Step 5b: System updates invoice as "Partially Paid"
    - Step 5c: Remaining balance tracked
- A2: Insurance Payment
    - Step 5a: Payment method = Insurance
    - Step 5b: System validates insurance details
    - Step 5c: System submits claim to insurance
    - Step 5d: Patient pays co-pay amount
- A3: Payment Failed
    - Step 7a: Payment gateway returns error
    - Step 7b: System displays error message
    - Step 7c: Actor retries or selects different method

Business Rules:

- BR-030: All payments must have transaction ID
- BR-031: Receipts must be issued for all payments
- BR-032: Refunds require manager approval
- BR-033: Cash payments > $1000 require verification

Use Case 5: Order Laboratory Test

Field	Description
Use Case ID	UC-005
Use Case Name	Order Laboratory Test
Primary Actor	Doctor
Secondary Actor	Lab Technician, Patient
Description	Order diagnostic tests for patient
Preconditions	Patient under doctor's care
Postconditions	Lab test ordered, patient notified
Trigger	Doctor determines need for tests

Main Flow:

1. Doctor accesses patient record
2. Doctor selects "Order Lab Test"
3. System displays available test catalog
4. Doctor selects test(s) to order
5. Doctor enters clinical notes/priority
6. System calculates estimated cost
7. Doctor confirms order
8. System generates test requisition
9. System notifies lab department
10. System notifies patient (instructions)
11. System updates patient record

Alternative Flows:

- A1: Urgent Test
    - Step 5a: Doctor marks as STAT/urgent
    - Step 5b: System alerts lab immediately
    - Continue from step 6

- A2: Package Test
    - Step 4a: Doctor selects test package (e.g., "Annual Physical")
    - Step 4b: System includes multiple tests
    - Continue from step 5

Business Rules:

- BR-040: Fasting tests require 8-12 hour fasting
- BR-041: STAT tests must be completed within 2 hours
- BR-042: Normal tests completed within 24-48 hours
- BR-043: Critical results require immediate doctor notification

6.2 Component Algorithms (PDL/Pseudocode)

This subsection provides the procedural logic for key service methods using Procedural Description Language (PDL). This ensures developers understand the exact flow, error handling, and data interactions.

Component: `AppointmentService`

Method: *`bookAppointment(AppointmentRequest req)`*

pdl

```
METHOD bookAppointment(AppointmentRequest req) RETURNS AppointmentResponse
    LOCAL vars:
        Patient patient
        Doctor doctor
        List<Appointment> existingAppts
        Appointment newAppt
        Boolean conflictFound

    // 1. Input Validation
    IF req.startTime IS NULL OR req.endTime IS NULL THEN
        RETURN ERROR("Invalid time range")
    END IF
    IF req.startTime >= req.endTime THEN
        RETURN ERROR("End time must be after start time")
    END IF

    // 2. Fetch Entities
    patient ← PatientRepository.findById(req.patientId)
    doctor ← DoctorRepository.findById(req.doctorId)

    IF patient IS NULL OR doctor IS NULL THEN
        RETURN ERROR("Patient or Doctor not found")
    END IF

    // 3. Check Availability (Optimistic Locking Strategy)
    // Query DB for overlapping appointments for this doctor on this date
    existingAppts ← AppointmentRepository.findOverlapping(
                        doctor.id,
                        req.startTime,
                        req.endTime
                    )

    IF existingAppts IS NOT EMPTY THEN
        RETURN ERROR("Time slot already booked")
    END IF

    // 4. Create Appointment Entity
    newAppt ← NEW Appointment()
```

```
    newAppt.setId(UUID.generate())
    newAppt.setPatient(patient)
    newAppt.setDoctor(doctor)
    newAppt.setStartTime(req.startTime)
    newAppt.setEndTime(req.endTime)
    newAppt.setStatus("SCHEDULED")
    newAppt.setCreatedAt(NOW())

    // 5. Persist within Transaction
    TRY
        BEGIN TRANSACTION
            AppointmentRepository.save(newAppt)
            // Update Doctor's cached availability in Redis
            CacheService.invalidate("doctor:" + doctor.id + ":slots:" + req.date)
        COMMIT TRANSACTION
    CATCH Exception e
        ROLLBACK TRANSACTION
        LOG.error("Failed to book appointment", e)
        RETURN ERROR("System error during booking")
    END TRY

    // 6. Async Notification
    EventPublisher.publish("AppointmentCreated", newAppt)

    RETURN SUCCESS(newAppt)
END METHOD
```

Component: `BillingService`

Method: *`generateInvoice(InvoiceRequest req)`*

```pdl
METHOD generateInvoice(InvoiceRequest req) RETURNS Invoice
    LOCAL vars:
        Appointment appt
        BigDecimal totalAmount
        BigDecimal taxAmount
        Invoice invoice

    // 1. Validate Appointment
    appt ← AppointmentRepository.findById(req.appointmentId)
    IF appt IS NULL OR appt.getStatus() != "COMPLETED" THEN
        RETURN ERROR("Cannot bill incomplete appointment")
    END IF

    // 2. Calculate Costs
    totalAmount ← appt.getConsultationFee()

    FOR EACH item IN req.additionalServices DO
        totalAmount ← totalAmount + item.getPrice()
    END FOR

    // 3. Apply Tax (if applicable)
    taxAmount ← totalAmount * TAX_RATE
    totalAmount ← totalAmount + taxAmount

    // 4. Create Invoice
    invoice ← NEW Invoice()
    invoice.setId(UUID.generate())
    invoice.setPatient(appt.getPatient())
    invoice.setTotalAmount(totalAmount)
    invoice.setTaxAmount(taxAmount)
    invoice.setStatus("PENDING")
    invoice.setIssueDate(TODAY)
    invoice.setDueDate(TODAY + 30 DAYS)

    // 5. Save and Return
    InvoiceRepository.save(invoice)
    RETURN invoice
END METHOD
```

Component: `MedicalRecordService`

Method: *`updateRecord(RecordUpdateDTO dto)`*

```pdl
METHOD updateRecord(RecordUpdateDTO dto) RETURNS MedicalRecord
    LOCAL vars:
        MedicalRecord record
        User currentUser

    // 1. Authorization Check
    currentUser ← SecurityContext.getCurrentUser()
    IF currentUser.getRole() != "DOCTOR" THEN
        RETURN ERROR("Unauthorized access")
    END IF

    // 2. Fetch Existing Record
    record ← MedicalRecordRepository.findById(dto.recordId)
    IF record IS NULL THEN
        RETURN ERROR("Record not found")
    END IF

    // 3. Update Fields
    record.setDiagnosis(dto.diagnosis)
    record.setIcdCode(dto.icdCode)
    record.setPrescription(dto.prescriptionJSON)
    record.setVitals(dto.vitalsJSON)
    record.setUpdatedAt(NOW())
    record.setUpdatedBy(currentUser.getId())

    // 4. Save with Versioning (for audit trail)
    MedicalRecordRepository.save(record)

    // 5. Audit Log
    AuditService.log(
        "MEDICAL_RECORD_UPDATED",
        currentUser.getId(),
        record.getId(),
        NOW()
    )

    RETURN record
END METHOD
```

6.3 Deployment Diagram

A. Deployment Diagram Description

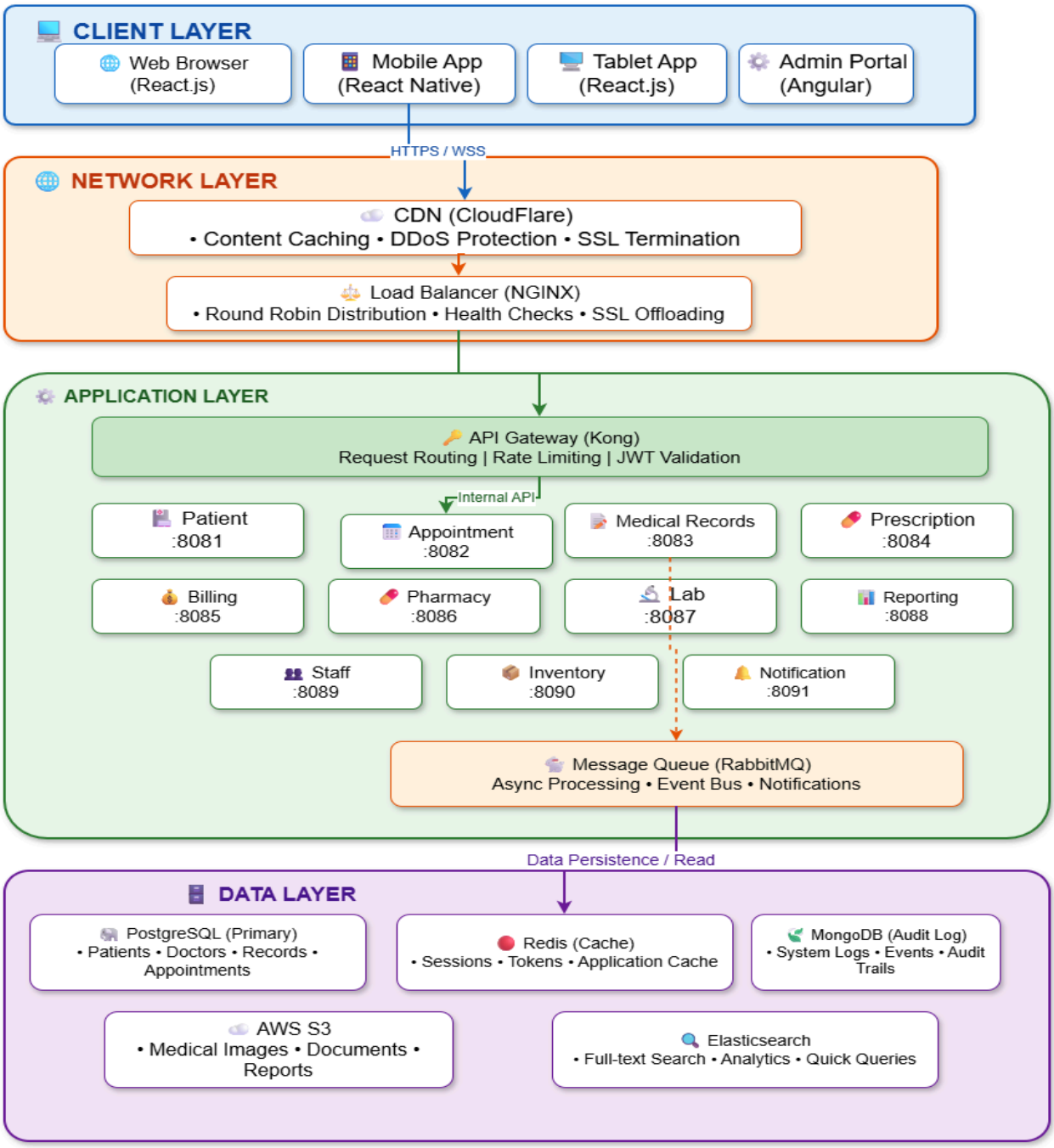
The HMS is deployed on a Kubernetes Cluster hosted on a cloud provider (AWS/Azure/GCP) or on-premise bare metal. The architecture follows a Multi-Tier Network Segmentation model for security.
Tiers:

1. Client Tier (Public Internet):
    - Users access via Web Browsers (Chrome/Firefox) or Mobile Apps.

    - Traffic enters through a Global Load Balancer (AWS ALB or NGINX Ingress).
2. DMZ Tier (Public Subnet):
    - API Gateway (Kong): Handles SSL termination, rate limiting, and JWT validation.
    - WAF (Web Application Firewall): Filters malicious traffic (SQLi, XSS).
3. Application Tier (Private Subnet - App Network):
    - Kubernetes Nodes: Run Docker containers for microservices.
        - `patient-service` (Replicas: 3)
        - `appointment-service` (Replicas: 3)
        - `billing-service` (Replicas: 2)
        - `auth-service` (Replicas: 2)
    - Service Mesh (Istio): Manages mTLS encryption between services and observability.
    - Message Broker (RabbitMQ): Deployed as a StatefulSet for async events.
4. Data Tier (Secure Subnet - Data Network):
    - PostgreSQL Primary: Read/Write operations.
    - PostgreSQL Replica: Read-only scaling.
    - Redis Cluster: Caching layer.
    - AWS S3 Bucket: Stores medical images (accessible via IAM roles only).
    - Elasticsearch: Stores audit logs.

Network Flow:
`Client` → `Load Balancer` → `API Gateway` → `K8s Service` → `Pod` → `PostgreSQL/Redis`

4.4 High-Level Architecture Diagram



B. Physical Node Mapping

Logical Component	Physical Resource	Quantity	Specs
Load Balancer	AWS ALB / NGINX	1 (Managed)	High Availability
App Servers	Kubernetes Worker Nodes	3-6	t3.xlarge (4 vCPU, 16GB RAM)
Database Primary	EC2 Instance / RDS	1	r5.large (2 vCPU, 16GB RAM, SSD)
Database Replica	EC2 Instance / RDS	1	r5.large (2 vCPU, 16GB RAM, SSD)
Cache Server	ElastiCache / VM	1	t3.medium (2 vCPU, 4GB RAM)
Storage	AWS S3 / EBS	Unlimited	Standard Storage Class

6.4 Environment Description

This section defines the hardware and software stack required to run the HMS in Production, Staging, and Development environments.

Database	Cache	Storage	Network
Development	1 Node (Local Docker)	1 Instance (4 CPU, 8GB)	1 Instance (2 CPU, 4GB)
Staging	2 Nodes (8 CPU, 16GB each)	2 Instances (HA, 8 CPU, 16GB)	2 Instances (4 CPU, 8GB)
Production	6 Nodes (16 CPU, 32GB each)	3 Instances (HA+DR, 16 CPU, 64GB)	3 Instances (8 CPU, 16GB)

B. Software Requirements

Column 1	Software	Version	Purpose
Operating System	Ubuntu Server	22.04 LTS	Host OS for all servers
Container Runtime	Docker	20.10+	Application containerization
Orchestration	Kubernetes	1.24+	Container management, scaling
Backend Framework	Java Spring Boot	3.1.x	Core business logic
Frontend Framework	React.js	18.x	Web Portal UI
Database	PostgreSQL	14.5+	Relational data storage
Cache	Redis	7.0+	Session & query caching
Message Queue	RabbitMQ	3.11+	Async event processing
Web Server	NGINX	1.22+	Reverse proxy, static files
API Gateway	Kong	3.0+	Routing, Auth, Rate Limiting
Monitoring	Prometheus + Grafana	Latest	Metrics collection & visualization
Logging	ELK Stack	8.x	Centralized log management
CI/CD	GitHub Actions	Latest	Automated build & deploy

C. Network Configuration

Segment	CIDR Block	Purpose	Access Rules
DMZ	10.0.1.0/12	Public-facing LB, API Gateway	Allow Inbound HTTPS (443) from Internet
App Network	10.0.2.0/12	K8s Pods, Microservices	Allow Inbound from DMZ; Allow Outbound to Data Network
Data Network	10.0.3.0/12	PostgreSQL, Redis, S3 Endpoints	Allow Inbound ONLY from App Network (Port 5432, 6379)
Management	10.0.4.0/12	Bastion Host, Monitoring	Allow SSH (22) from Corporate IP only

Security Groups / Firewall Rules:

1. DB Security Group: Allow Port 5432 ONLY from `10.0.2.0/12` (App Network).
2. App Security Group: Allow Port 8080 ONLY from `10.0.1.0/12` (DMZ).
3. Redis Security Group: Allow Port 6379 ONLY from `10.0.2.0/12`.
4. SSH Access: Restricted to Bastion Host in Management Network.

CHAPTER 7. REQUIREMENTS MATRIX

This section provides a bidirectional traceability matrix linking the Software Requirements Specification (SRS) to the architectural components, data structures, and quality tactics defined in this document. It also records the critical design decisions, alternatives considered, and justifications for the chosen architecture.

.1 Traceability Matrix (SRS → Architecture)

The following table maps Functional Requirements (FR) and Non-Functional Requirements (NFR) to their corresponding system components, data entities, and verification methods.

7.1 Traceability Matrix (SRS → Architecture)

Cross-reference mapping functional requirements (FR) and non-functional requirements (NFR) to architectural components, data structures, and quality tactics.

SRS ID	CRequirement Description	Architecture Component(s	Data Structure(s)	Quality Tactic Applied	Verification Method
FR-01	Patient Registration	`PatientController`, `PatientService`, `PatientRepository`	`patients` table, `users` table	Validation, Idempotent creation	Unit/Integration Tests
FR-02	Appointment Booking	`AppointmentService`, `Scheduler`, `WebPortal`	`appointments` table	Conflict detection, Caching	Load Testing, UI Testing
FR-03	Medical Record Management	`MedicalRecordService`, `AuditLogger`, `S3Storage`	`medical_records` (PostgreSQL + JSONB)	RBAC, Audit Logging, Encryption	Pen Testing, Code Review
FR-04	Prescription Generation	`PrescriptionService`, `PharmacyUI`	`prescriptions`, `medications`	Digital Signature, Validation	Functional Testing
FR-05	Lab Test Ordering & Results	`LabService`, `RabbitMQPublisher`, `LabUI`	`lab_tests` table	Async Processing, Event Bus	Integration Testing
FR-06	Billing & Invoicing	`BillingService`, `PaymentGateway`, `InvoiceGen`	`invoices`, `payments`	ACID Transactions, Idempotency	Transaction Testing
NFR-01	Response Time < 2s	API Gateway,	Indexed columns,	Caching, Connection	JMeter Load

		Redis Cache, Read Replicas	Cached queries	Pooling	Testing
NFR-02	99.9% Availability	Kubernetes HPA, DB Replication, LB	Multi-AZ deployment	Redundancy, Failover, Health Checks	Chaos Testing, Uptime Monitoring
NFR-03	HIPAA Compliance	JWTAuthFilter, AuditLogger, TLS	`audit_logs`, `encrypted fields`	Encryption (at rest/transit), RBAC	Compliance Audit, Pen Testing
NFR-04	Horizontal Scalability	Microservice-ready containers, K8s	Stateless services, Shared DB	Stateless design, Service discovery	Scale-out Testing
NFR-05	Maintainability	Layered architecture, CI/CD, SonarQube	Modular packages, DTOs	Separation of Concerns, Automated Testing	Static Analysis, Code Coverage

7.2 Design Decisions & Trade-offs

This subsection documents the critical architectural choices made during the design phase, including alternatives considered, trade-offs analyzed, and the final justification.

Decision 1: Architectural Style

- Alternatives Considered: Monolithic Architecture vs. Microservices Architecture vs. Layered (N-Tier) Architecture.
- Trade-offs:
    - *Monolith:* Faster initial development but difficult to scale and maintain as complexity grows.

    - *Microservices:* Highly scalable and deployable independently but introduces significant operational complexity (service mesh, distributed tracing, network latency).
    - *Layered:* Balanced approach. Clear separation of concerns, easier testing, and modular enough to extract services later if needed.
- Final Decision & Justification: Layered Architecture.
    - *Justification:* Given the team size (5–10 developers) and the need for rapid MVP delivery, a Layered Architecture provides the best balance. It offers clear separation of concerns (Presentation, Application, Data) without the DevOps overhead of microservices. The modular design allows specific modules (e.g., Billing) to be extracted into microservices in Phase 2 if scaling demands it.

Decision 2: Database Technology

- Alternatives Considered: PostgreSQL vs. MySQL vs. MongoDB.
- Trade-offs:
    - *MySQL:* Good performance but weaker support for complex JSON data and GIS features compared to PostgreSQL.
    - *MongoDB:* Excellent for flexible schema (EMR) but lacks strong ACID compliance required for billing and financial transactions.
    - *PostgreSQL:* Strong ACID compliance, excellent JSONB support for semi-structured medical data, and robust indexing.
- Final Decision & Justification: PostgreSQL.
    - *Justification:* Medical and financial data require strict integrity (ACID). PostgreSQL's JSONB capability allows us to store flexible EMR data (diagnoses, vitals) without sacrificing relational integrity for core entities (Patients, Appointments, Invoices). It is also HIPAA-compliant with proper encryption.

Decision 3: Authentication Mechanism

- Alternatives Considered: Session-Based Authentication vs. OAuth2 vs. JWT (JSON Web Tokens).
- Trade-offs:
    - *Session-Based:* Secure but requires server-side session storage, making horizontal scaling difficult.
    - *OAuth2:* Industry standard but complex to implement for internal systems.

    - *JWT:* Stateless, easy to scale horizontally, and native support for mobile apps. Requires careful management of token expiration and refresh.
- Final Decision & Justification: JWT + Refresh Tokens.
    - *Justification:* JWTs enable stateless authentication, allowing any application instance to validate users without querying a central session store. This is critical for horizontal scaling in Kubernetes. Short-lived Access Tokens (15 mins) combined with long-lived Refresh Tokens (7 days) balance security and user experience.

Decision 4: Communication Pattern

- Alternatives Considered: Synchronous REST vs. gRPC vs. Asynchronous Messaging (RabbitMQ/Kafka).
- Trade-offs:
    - *gRPC:* High performance but tight coupling and harder debugging.
    - *Async Only:* Decoupled but poor user experience for immediate actions (e.g., login, booking).
    - *Hybrid:* Combines immediate feedback for user actions with decoupling for background tasks.
- Final Decision & Justification: Hybrid (REST + RabbitMQ).
    - *Justification:* Critical user-facing flows (Login, Appointment Booking, Payment) use Synchronous REST to provide immediate feedback. Background tasks (Email Notifications, Lab Result Processing, Report Generation) use Asynchronous RabbitMQ to decouple services, improve resilience, and prevent blocking the main thread.

Decision 5: Caching Strategy

- Alternatives Considered: No Cache vs. Memcached vs. Redis.
- Trade-offs:
    - *Memcached:* Simple and fast but limited to simple key-value pairs.
    - *Redis:* Supports complex data structures (Lists, Sets, Hashes), persistence, and pub/sub messaging.
- Final Decision & Justification: Redis.
    - *Justification:* Redis is versatile. It is used for:
        1. Session Storage: Storing JWT blacklists.
        2. Query Caching: Caching doctor availability slots to reduce DB load.

Pub/Sub: Real-time notifications for appointment confirmations.

3. Its rich feature set outweighs the slight complexity increase over Memcached.

Decision 6: Deployment Environment

- Alternatives Considered: Bare-Metal Servers vs. Virtual Machines (VMs) vs. Container Orchestration (Kubernetes).
- Trade-offs:
  - *Bare-Metal/VMs:* Simpler setup but inflexible scaling and longer deployment times.
  - *Kubernetes:* Steeper learning curve but enables auto-scaling, self-healing, and zero-downtime deployments.
- Final Decision & Justification: Kubernetes (K8s).
  - *Justification:* K8s provides Horizontal Pod Autoscaling (HPA) to handle peak loads (e.g., Monday morning appointments) automatically. It ensures high availability by restarting failed containers and supports Blue-Green Deployments for zero-downtime updates, which is critical for a 24/7 hospital system.

Decision Area	Alternatives Considered	Trade-offs	Final Decision & Justification
Decision Area	**Alternatives Considered**	**Trade-offs**	**Final Decision & Justification**
Architecture Style	Monolithic vs Microservices vs Layered	Monolith: fast dev but hard to scale. Microservices: highly scalable but complex ops.	Layered Architecture chosen. Balances rapid delivery, clear separation of concerns, and future microservice migration path without initial DevOps overhead.

Database	PostgreSQL vs MySQL vs MongoDB	MySQL: good for web but weaker JSON/GIS. MongoDB: flexible schema but lacks ACID for billing.	PostgreSQL chosen. Strong ACID compliance, excellent JSONB for flexible medical data, robust indexing, and HIPAA-ready encryption.
Authentication	Session-based vs OAuth2 vs JWT	Session: secure but hard to scale. OAuth2: standard but complex setup. JWT: stateless, mobile-friendly, requires token refresh strategy.	JWT + Refresh Tokens chosen. Enables stateless horizontal scaling, works seamlessly across web/mobile, and integrates with Spring Security.
Communication Pattern	Synchronous REST vs gRPC vs Async Messaging	gRPC: high perf but tight coupling. Async: decoupled but harder debugging. REST: universal, tooling-rich.	REST (sync) + RabbitMQ (async) hybrid. Critical user flows (booking, billing) use sync REST for immediate feedback. Notifications, reports, and lab updates use async messaging for decoupling.

.